Mehran University of Engineering and Technology, Jamshoro
Department of Software Engineering

Software Construction and Development (SW315)
Lab 16 — Open-Ended Lab: Complex Engineering Problem (CEP)
5th Semester

CampusCare

Smart Campus Service and Maintenance Management System

Group Members

Name	Roll Number
Shariyar	24SW028
Saqib Anwar	24SW046
Bansi	24SW052
Safiullah	24SW107
Saifullah	24SW137
Janak	24SW155

Course instructor: [to be filled in by the group]
Source repository: [to be inserted after the group pushes the project]
Submission deadline: 10-11-2026

Abstract

Reporting faults on a university campus is usually informal, which causes lost complaints, duplicate reports, unclear accountability and safety problems that wait in the same queue as cosmetic requests. This report describes CampusCare, a web-based service and maintenance request system built for the SW315 Lab 16 Complex Engineering Problem. Students and faculty report problems, maintenance staff resolve assigned tasks and administrators triage, assign, monitor and report, all under role-based access control.

We analysed the problem against the CEP characteristics, identified stakeholder conflicts (speed versus capacity, safety versus queue order, privacy versus accountability) and turned each into an explicit rule: a validated workflow state machine, target resolution times, safety-based priority escalation that preserves the reporter's own priority, duplicate detection with reporter override, workload-aware assignment hints, and privacy-limited views. Two architectures were compared, a Flask monolith with SQLite and a separated single-page application with API services; the monolith was chosen for scope, security simplicity and time, with its scalability limits documented.

The Flask prototype has 25 routes, six database tables, an audit log, and use case, class, sequence, state, ER and architecture diagrams generated alongside editable Draw.io files. A pytest suite of 37 items covering 35 numbered test cases, including negative and security cases, passed in the final run; the first run had three failures that came from the test set-up rather than from the application, as explained in the report. A CI workflow is provided but has not yet been run on GitHub. The report ends with limitations, risks and improvements, and lists the evidence the group still has to produce.

Table of Contents

1. Introduction..... 5
1.1 Background..... 5
1.2 Problem statement..... 5
1.3 Objectives..... 5
1.4 Scope..... 5
2. Complex Engineering Problem Analysis..... 6
3. Stakeholder Analysis..... 7
3.1 Conflicting needs and how the system addresses them..... 7
4. Software Requirements Specification..... 8
4.1 Purpose, scope and users..... 8
4.2 Functional requirements..... 8
4.3 Non-functional requirements..... 9
4.4 Constraints, assumptions and risks..... 9
4.5 External interfaces and data..... 10
4.6 Requirements traceability..... 10
5. Engineering Alternatives and Decision-Making..... 11
5.1 The two approaches..... 11
5.2 Decision and trade-off..... 12
5.3 Smaller design decisions..... 12
6. Software Design..... 13
6.1 System architecture..... 13
6.2 Use case diagram..... 14
6.3 Class diagram..... 15
6.4 Sequence diagrams..... 17
6.5 Workflow state diagram..... 19
6.6 Database design..... 20
7. Implementation..... 21
7.1 Technology and structure..... 21
7.2 Key mechanisms..... 22
8. User Interface and Main Features..... 22
9. Development and DevOps..... 29
9.1 Version control..... 29
9.2 Continuous integration..... 30
9.3 Deployment considerations..... 30
10. Testing and Evaluation..... 30
10.1 Methodology..... 30
10.2 Results..... 30
10.3 Interpretation..... 33
10.4 Security testing considerations..... 33
10.5 Limitations, failure risks and improvements..... 34
10.6 Tests not executed..... 34
11. System Evaluation..... 34
12. Compliance with Lab 16 and Assessment Criteria..... 35
13. Limitations and Future Improvements..... 36
14. Lessons Learned and Team Contributions..... 36
14.1 Lessons learned..... 36
14.2 Team and proposed responsibilities..... 37
15. Conclusion..... 37

References.....37
Appendix A: All Test Cases and Recorded Results.....38
Appendix B: Running the Project.....39
Appendix C: Evidence the Group Must Still Produce.....40

1. Introduction

1.1 Background

A university campus is a collection of buildings, laboratories, classrooms, libraries, washrooms and electrical and network installations that all need continuous maintenance. In our department the usual way of reporting a fault is informal: a message to a class representative, a phone call, or a visit to the administration office. Nothing records who reported what, who was told, or whether anyone fixed it. Two students can report the same projector failure and nobody notices the duplicate; a genuine safety hazard can sit in the same pile as a request to repaint a wall.

CampusCare is our answer to that problem: a small web application in which students and faculty report problems, maintenance staff receive and resolve assigned tasks, and administrators triage, assign and monitor everything from one place. The project was chosen for Lab 16 because it has real interacting requirements and real conflicts between stakeholders, which is what the lab's Complex Engineering Problem (CEP) brief asks for, rather than being a simple create-read-update-delete exercise.

1.2 Problem statement

Design, build and evaluate a centralised campus service-request system that (a) lets reporters describe a fault precisely and follow its progress, (b) gives administrators a defensible way to prioritise and assign scarce maintenance capacity, (c) gives staff enough information to act without chasing the reporter, and (d) keeps an accountable record of every decision while limiting how much personal information each role can see.

1.3 Objectives

- Provide a complete request life-cycle (Submitted → Under Review → Assigned → In Progress → Resolved → Closed, plus Rejected) with enforced valid transitions and role permissions.
- Resolve the main stakeholder conflicts with explicit, explainable rules (priority escalation, target resolution times, workload-aware assignment, privacy-limited views) instead of leaving them to ad-hoc judgement.
- Compare at least two architectural approaches against the requirements and justify the one selected, including its limits.
- Produce consistent UML models, a normalised database and a working, tested prototype kept in version control with a CI workflow.
- Evaluate the prototype honestly with positive and negative tests and report what is not yet covered.

1.4 Scope

In scope: the three roles, the request workflow, duplicate detection, priority handling, image upload, feedback, dashboards, reports, user management and audit logging. Out of scope for this lab: e-mail/SMS notifications, integration with university identity systems, mobile native apps, asset inventory and spare-parts management, and load testing at campus scale. These exclusions keep the prototype achievable within the lab period; Section 13 explains what each exclusion would cost in practice.

2. Complex Engineering Problem Analysis

The Lab 16 brief lists nine characteristics of a complex engineering problem and marks eight of them as applicable to this assignment. We checked each against CampusCare and only claim a characteristic where we can point to a concrete example in the project. One characteristic (diverse stakeholder groups) is not claimed, in line with the brief.

Table 1: CEP characteristics and where they appear in CampusCare

Characteristic	Applies	Evidence from the project
1. Depth of knowledge required	Yes	The solution combines requirements engineering, state-machine design, relational modelling with constraints, web security (CSRF, RBAC, upload validation, password storage), UML, testing and CI. A correct design needs all of them at once; for example the upload feature is simultaneously a usability, security and storage-design problem.
2. Range of conflicting requirements	Yes	Students want immediate resolution; administrators have limited staff; staff want complete information before accepting work; safety issues must jump the queue; privacy must be protected while every action stays accountable (Section 3 lists the conflicts and the rules used).
3. Depth of analysis required	Yes	There is no single correct answer to “what priority is this?” or “is this a duplicate?”. We had to define heuristics, test them against edge cases (e.g. “fire extinguisher” must not mean fire) and analyse their failure modes (RISK-01, RISK-06).
4. Infrequently encountered issues	Yes	Re-opened requests, a request reassigned while in progress, the same fault reported twice by different people, a reporter whose chosen priority differs from the real urgency, lock-out versus account enumeration, and files whose extension and content disagree.
5. Beyond codes / standards of practice	Partly	Standards (ISO/IEC/IEEE 29148 for requirements, OWASP guidance for security) help but do not say how a campus should rank a leaking tap against a flickering light, or what staff may see about a reporter. These rules are our own engineering judgement and are exposed as configurable constants.
6. Diverse groups of stakeholders	No	Three internal roles with clear responsibilities. There are no external contractors, public users or regulators in the prototype, so we do not claim this characteristic.
7. Interdependence of parts	Yes	Workflow rules, permissions, assignment, SLA timers, duplicate detection, audit logging and reporting all depend on the same request data. Changing the transition table changes what the UI offers, what the tests expect and what the reports count.
8. Significant consequences	Yes	A wrongly ranked sparking socket or a lost gas-leak report has safety consequences; a privacy leak affects individuals; an unresponsive system undermines trust in the service. Consequences differ by context (hostel, laboratory, library).
9. Judgement in decision-making	Yes	Architecture choice, escalation rules, what to log, how strict to make uploads, and what to leave out of the prototype all required trade-off decisions that are documented in Sections 5 and 13.

3. Stakeholder Analysis

Table 2: Stakeholder groups and their needs

Stakeholder	Needs	Concerns / constraints
Students and faculty (reporters)	Quick, simple reporting; proof the report was received; visible progress; a way to say the problem is not fixed.	Do not want to retype the same report; expect fast action; do not want contact details shown widely.
Maintenance staff	A prioritised list of assigned tasks; location, description and photo; ability to record what was done.	Limited hours; incomplete reports waste visits; should not be blamed for tasks that were never assigned to them.
Administrators (estate / department office)	Overview of all requests, overdue items and workload; control of assignment and priority; reports for management.	Limited staff and budget; must justify priority decisions; must manage user accounts safely.
University management (indirect)	Evidence of service quality (resolution time, satisfaction).	Accountability and data protection.

3.1 Conflicting needs and how the system addresses them

Table 3: Conflicts and the design response

Conflict	How CampusCare responds
Students expect immediate resolution; administrators have limited maintenance resources.	Every priority level has an explicit target resolution time (Critical 8 h, High 24 h, Medium 72 h, Low 168 h). The reporter sees the target and the assignee; the administrator sees overdue items and each staff member's open-task count, so expectations and capacity are both visible.
Staff need sufficient information before accepting a task.	Title, 20-character minimum description, building, specific location, category and an optional photo are mandatory or supported at submission. Staff can read the administrator's assignment note. Rejection requires a written reason, so incomplete requests are bounced with an explanation rather than ignored.
Safety issues must precede ordinary complaints.	Reporters choose a priority, but the system keeps that choice as `requested_priority` and computes an effective `priority`: Safety & Security is at least High, and danger keywords (e.g. sparking, smoke, gas leak) force Critical. Queues sort by effective priority. Administrators can lower a priority only with a recorded reason.
The university must protect user data and also keep accountability.	Staff see the reporter's first name only; contact e-mail is shown to administrators. Passwords are stored as salted hashes. Every state change, assignment, priority change, login failure and export writes an audit row with user, time and IP address. Uploaded photos are served only to people allowed to see the request.
Reporters want to report freely; administrators do not want duplicates.	Duplicate detection warns the reporter (live and on submit) and lets them continue if the problem is different. The confirmed duplicate is flagged for the administrator instead of being silently dropped.

4. Software Requirements Specification

4.1 Purpose, scope and users

This section follows the structure recommended by ISO/IEC/IEEE 29148 in condensed form. CampusCare is a browser-based system used on a university network. It has three user roles; the permissions in Table 4 are enforced on the server for every request, not only by hiding buttons.

Table 4: Role permissions

Capability	Student / Faculty	Maintenance staff	Administrator
Register / log in	Yes (self-register)	Yes (created by admin)	Yes (created by admin)
Submit request, upload photo	Yes	No	No
View requests	Own only	Assigned to them	All
Start review, reject, assign, re-assign	No	No	Yes
Start work, resolve with notes	No	Assigned tasks only	No
Close or re-open a resolved request	Own request	No	Yes
Change priority	No	No	Yes (reason required)
Give feedback	Own, after resolution	No	No
Statistics, reports, CSV export	No	No	Yes
Manage users, view audit log	No	No	Yes

4.2 Functional requirements

Table 5: Functional requirements

ID	Requirement	Implemented in
FR-01	Users shall register (reporters only), log in and log out with securely hashed passwords and session management.	routes/auth.py, models.User
FR-02	The system shall enforce role-based access control for Student/Faculty, Maintenance Staff and Administrator.	routes/__init__.py (role_required)
FR-03	A reporter shall submit a request with title, description, category, building, location, priority and an optional PNG/JPG image, with input validation.	services.create_request
FR-04	Reporters shall track status, history and assigned staff of their requests.	requests/detail.html
FR-05	The system shall enforce the workflow Submitted → Under Review → Assigned → In Progress → Resolved → Closed, with Rejected, valid transitions and role permissions.	services.TRANSITIONS, transition()
FR-06	Administrators shall assign and re-assign requests to active maintenance staff, with workload and specialty hints.	services.assign, suggest_staff
FR-07	Staff shall view assigned tasks, update status, add resolution notes and view task history.	routes/staff.py
FR-08	Administrators shall view, search and filter all requests (category, priority, location/building, status, overdue) and change priority with a reason.	services.apply_filters, change_priority
FR-09	The system shall detect probable duplicate requests using building, category and location.	services.find_duplicates

ID	Requirement	Implemented in
FR-10	The system shall handle requests by priority and escalate safety hazards automatically.	services.escalate_priority
FR-11	Reporters shall submit one rating (1–5) and comment after resolution.	services.submit_feedback
FR-12	Administrators shall see dashboard statistics and overdue requests.	services.dashboard_stats, Chart.js
FR-13	Administrators shall generate maintenance reports for a date range and export CSV.	services.report_summary, report_csv
FR-14	Administrators shall create users, change roles and (de)activate accounts, but not their own.	routes/admin.py
FR-15	The system shall record request/assignment history and an audit log.	RequestUpdate, AuditLog, services.audit

4.3 Non-functional requirements

Table 6: Non-functional requirements

ID	Requirement	How it is met / verified
NFR-01	Security: hashed passwords, lock-out after 5 failed logins for 15 minutes, CSRF protection on all POSTs, restrictive response headers, validated uploads with random names.	Implemented; TC-02, 03, 04, 05, 10, 17–19
NFR-02	Usability: responsive layout, clear status indicators, empty states and understandable error messages.	Manual review and screenshots (Section 8)
NFR-03	Performance: ordinary pages should respond in well under one second with demonstration-sized data.	Measured mean 5–7 ms per page (Section 11); not a load test
NFR-04	Maintainability: layered structure (routes → services → models), no business rules in templates, automated tests.	Code structure; 37 pytest tests
NFR-05	Portability: runs locally with Python 3 and SQLite; no external services; database URL configurable.	README; config.py
NFR-06	Privacy: staff see reporters' first names only; photos only to authorised viewers.	TC-09, TC-17
NFR-07	Data integrity: CHECK, UNIQUE and foreign-key constraints in the schema.	models.py; Section 6.6

4.4 Constraints, assumptions and risks

Table 7: Constraints

ID	Constraint
CR-01	The prototype must run on a student laptop with no paid or external services (Python, Flask, SQLite).
CR-02	The system must be built and tested by a group of six within the lab period, so scope was limited to the items in Section 1.4.
CR-03	Only demonstration data is used; no real university or personal data.
CR-04	Images are limited to PNG/JPG, 2 MB each.

Assumptions. Users reach the system over a trusted campus network or VPN; each user has a unique e-mail address; one administrator team is responsible for triage; maintenance staff are assigned by the administrator rather than self-selecting tasks.

Table 8: Risks

ID	Risk	Mitigation / residual risk
RISK-01	Keyword escalation produces false positives (e.g. “fire drill” or “fire door”) and floods the Critical queue.	Rule excludes “fire extinguisher”; administrators can lower priority with a reason; keyword list is a constant that can be tuned. Residual: still crude compared with human reading.
RISK-02	SQLite allows limited concurrent writes.	Acceptable for the prototype; DATABASE_URL allows PostgreSQL/MySQL for real deployment.
RISK-03	Staff capacity cannot meet the target times, so many requests become overdue.	Overdue list and workload counts make this visible; targets are constants to be agreed with the estate office.
RISK-04	Account takeover through weak or guessed passwords.	Password rules, lock-out, hashing. Residual: no two-factor authentication.
RISK-05	Malicious or disguised uploads.	Extension, size and magic-byte checks; random names; access-controlled download. Residual: images are not re-encoded or virus-scanned.
RISK-06	Duplicate heuristic misses differently worded locations or flags different problems in the same room.	Reporter can override; administrator sees the flag. Residual: free-text locations are inherently ambiguous.
RISK-07	No notifications, so reporters or staff may not notice changes.	Documented limitation; future work.

4.5 External interfaces and data

User interface: server-rendered HTML pages (Jinja2, Bootstrap 5) with small ES6 scripts. **HTTP interfaces:** form posts for all state changes plus two read-only JSON endpoints (`/api/duplicates` for the live duplicate hint and `/admin/api/stats` for the charts). **Storage:** a SQLite database and an uploads directory under `instance/`. The database requirements are described in Section 6.6 and in `docs/DATABASE.md`.

4.6 Requirements traceability

Table 9: Traceability from requirements to implementation and test cases

Req.	Main implementation	Test cases
FR-01	auth.py, User model	TC-01, 02, 05, 34
FR-02	role_required, can_view	TC-01, 04, 06, 07, 08, 09
FR-03	create_request, validate_image	TC-11, 12, 17, 18, 19
FR-04	detail view, history	TC-25 (manual screenshots)
FR-05	TRANSITIONS, transition()	TC-20, 21, 23, 25, 26, 27
FR-06	assign(), suggest_staff()	TC-08, 20, 22, 33
FR-07	staff routes, transition()	TC-23, 24, 25
FR-08	apply_filters, change_priority	TC-29, 30, 31

Req.	Main implementation	Test cases
FR-09	find_duplicates	TC-15, 16
FR-10	escalate_priority, SLA_HOURS	TC-13, 14, 30, 31
FR-11	submit_feedback	TC-28
FR-12	dashboard_stats, charts.js	TC-30 (logic); charts verified by screenshot only
FR-13	report_summary, report_csv	TC-32
FR-14	admin user routes	TC-34, 35
FR-15	RequestUpdate, audit()	TC-11, 20
NFR-01	auth, CSRF hook, upload checks	TC-02, 03, 04, 05, 10, 17, 18, 19
NFR-06	can_view, uploaded_file()	TC-09, 17

5. Engineering Alternatives and Decision-Making

5.1 The two approaches

Approach A — server-rendered monolith. One Flask application renders HTML pages, talks to SQLite through SQLAlchemy, and stores uploads on local disk. The browser receives finished pages; JavaScript is used only for small enhancements.

Approach B — separated front end and API services. A single-page application (for example React) calls a REST API implemented as one or more services, backed by a server database such as PostgreSQL, with token-based authentication and object storage for images.

Table 10: Comparison of the two approaches

Criterion	A: Flask monolith + SQLite	B: SPA + API + PostgreSQL
Development complexity	Low: one codebase, one language, forms and templates.	High: two codebases, API contract, client state management, CORS.
Performance	Fast for hundreds of users; page reloads on every action.	Smoother interaction; extra network round trips for data.
Scalability	Limited by one process and SQLite writes; can move to PostgreSQL and several workers later.	Scales horizontally by design.
Security	Smaller attack surface; sessions and CSRF handled server-side.	Larger surface: token storage, CORS, API authorisation on every endpoint.
Maintenance	Easy for a student team to understand and test.	More moving parts; needs more discipline and tooling.
Cost	None beyond a small server.	Database server, object storage, build pipeline.
Deployment	Single process; copy and run.	Several deployables, environment configuration, reverse proxy.
Suitability for the university	A department-level service with tens to a few hundred users.	A campus-wide product with mobile apps and many integrations.
Time constraints	Achievable within the lab period.	Would consume the period on infrastructure rather than on the engineering problem.

5.2 Decision and trade-off

We selected **Approach A**. The decisive factors were time, the size of the user base and security simplicity: the engineering problem in this assignment lies in the workflow, prioritisation and access rules, not in front-end rendering or horizontal scaling, and a monolith lets the whole group read, run and test all of the code. We structured it in layers (routes → services → models) so that the business rules do not depend on Flask and can later sit behind an API if Approach B becomes necessary.

Main trade-off: simplicity and low operating cost against scalability and interactivity. SQLite serialises writes and a single process limits throughput, and each action reloads a page. For a department with a few hundred users and a handful of staff this is acceptable. For a whole-campus roll-out we would not make this choice again without changes: PostgreSQL, several WSGI workers, a task queue for notifications, and probably an API for a mobile client. Approach A is therefore appropriate for this project's scope, not universally better.

5.3 Smaller design decisions

Table 11: Other decisions considered

Decision	Chosen	Why / alternative rejected
Where to keep assignments	Separate `request_assignments` table with an active flag	A single `assigned_to` column is simpler but loses re-assignment history and cannot show who assigned whom.
Priority handling	Store requested and effective priority; rule-based escalation	Trusting the reporter fails for safety; machine-learning classification needs data we do not have and is hard to explain to an administrator.
Workflow rules	One transition table checked in one function	Scattering `if status ==` … checks across routes makes rules inconsistent and hard to test.
Upload storage	Files outside `static/`, served by a permission-checked route	Public static serving is simpler but exposes every photo to anyone who guesses a URL.
CSRF protection	Small session-token check in the application	Flask-WTF would add a dependency for one feature; the hook is about 10 lines and fully tested (TC-10).
Front-end libraries	Bootstrap and Chart.js vendored in `static/vendor`	CDN links break offline and in restricted networks; vendoring keeps CR-01 true.

6. Software Design

6.1 System architecture

CampusCare has three tiers inside one deployable unit. The browser renders Jinja2 pages and runs small scripts. A single Flask process contains thin route controllers, a service layer holding every business rule, and SQLAlchemy models. Data lives in SQLite and an uploads folder. Cross-cutting security (sessions, role checks, CSRF, response headers) sits in front of all routes. Figure 1 shows the structure; it is the same layering that the class diagram in Section 6.3 documents in detail.

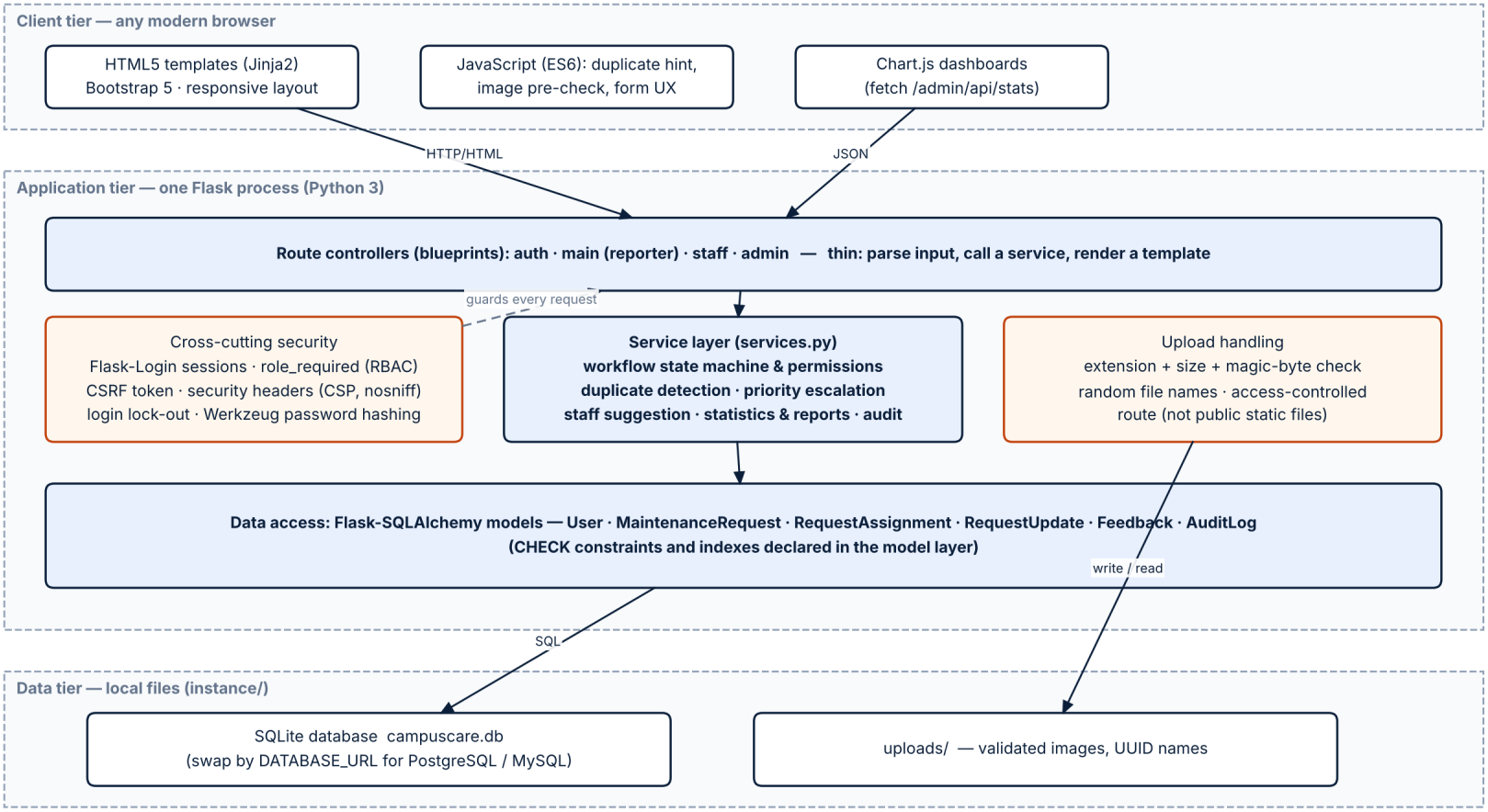


Figure 1: System architecture of CampusCare

6.2 Use case diagram

The use case diagram (Figure 2) lists what each actor can do. It shows the three actors and their use cases, the include relationships for duplicate detection and input validation inside “Submit Maintenance Request”, the optional image upload (extend), and the resolution notes included in “Update Task Status”. Every use case corresponds to a functional requirement in Table 5 and to an implemented screen.

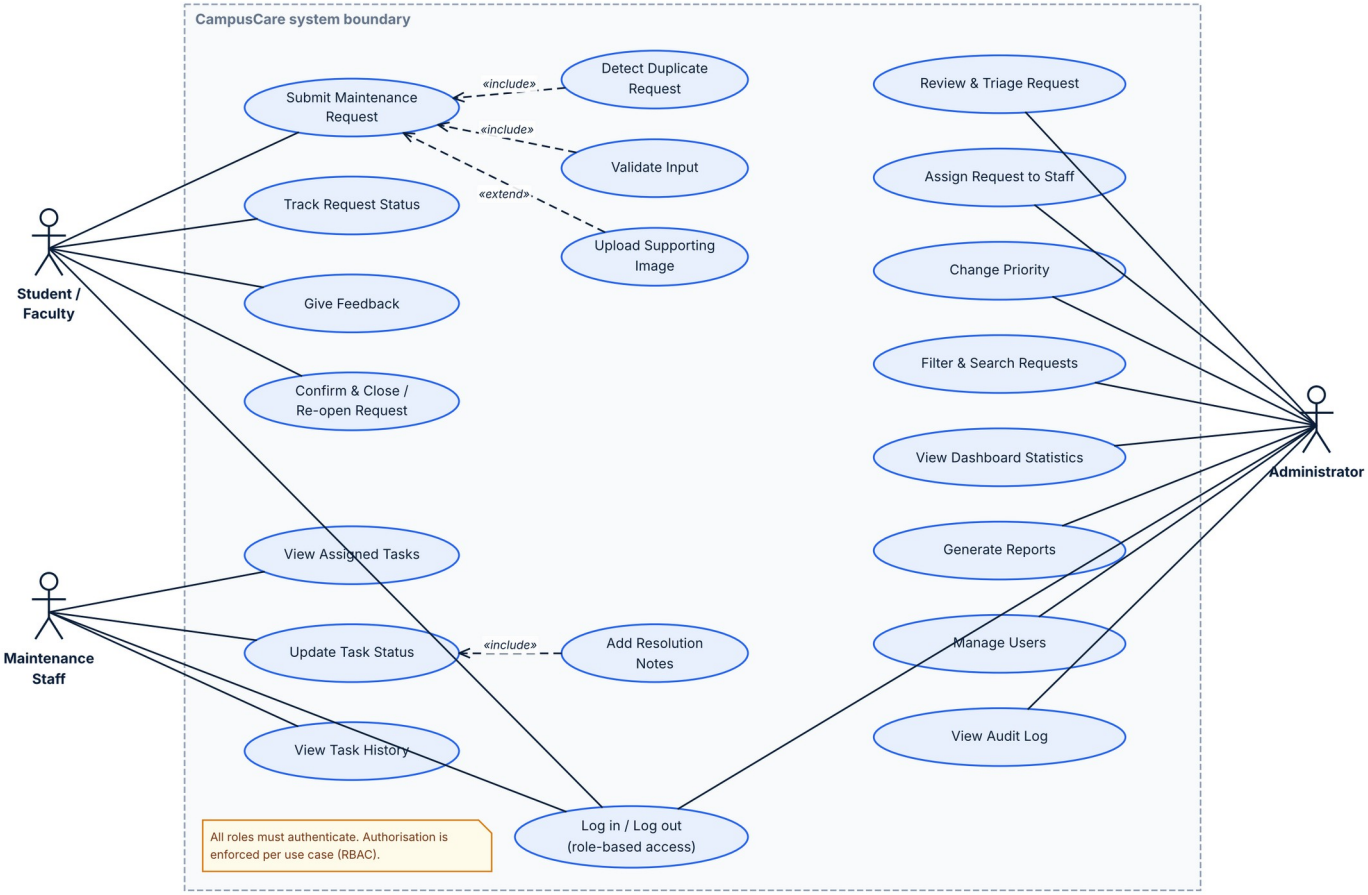


Figure 2: Use case diagram

6.3 Class diagram

The class diagram (Figures 3 and 4) shows the real application structure rather than an idealised domain model. Figure 3 covers the presentation and business layers: four blueprints, the RBAC and security hooks, the two service groups (request actions and queries) and the workflow policy. Figure 4 covers the six persistence classes with their attributes, key relationships and multiplicities. Composition is used for assignments and history because they cannot exist without their request.

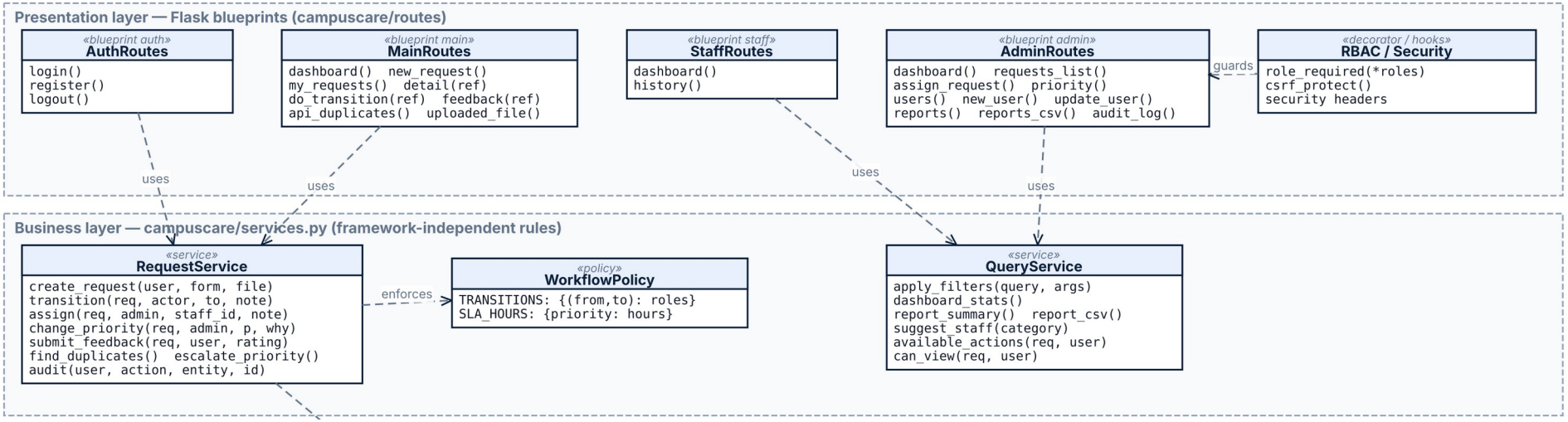


Figure 3: Class diagram (1/2): presentation and business layers

Figure 4 continues the class diagram with the persistence layer. Attribute lists follow the implemented models; multiplicities are shown at the association ends.

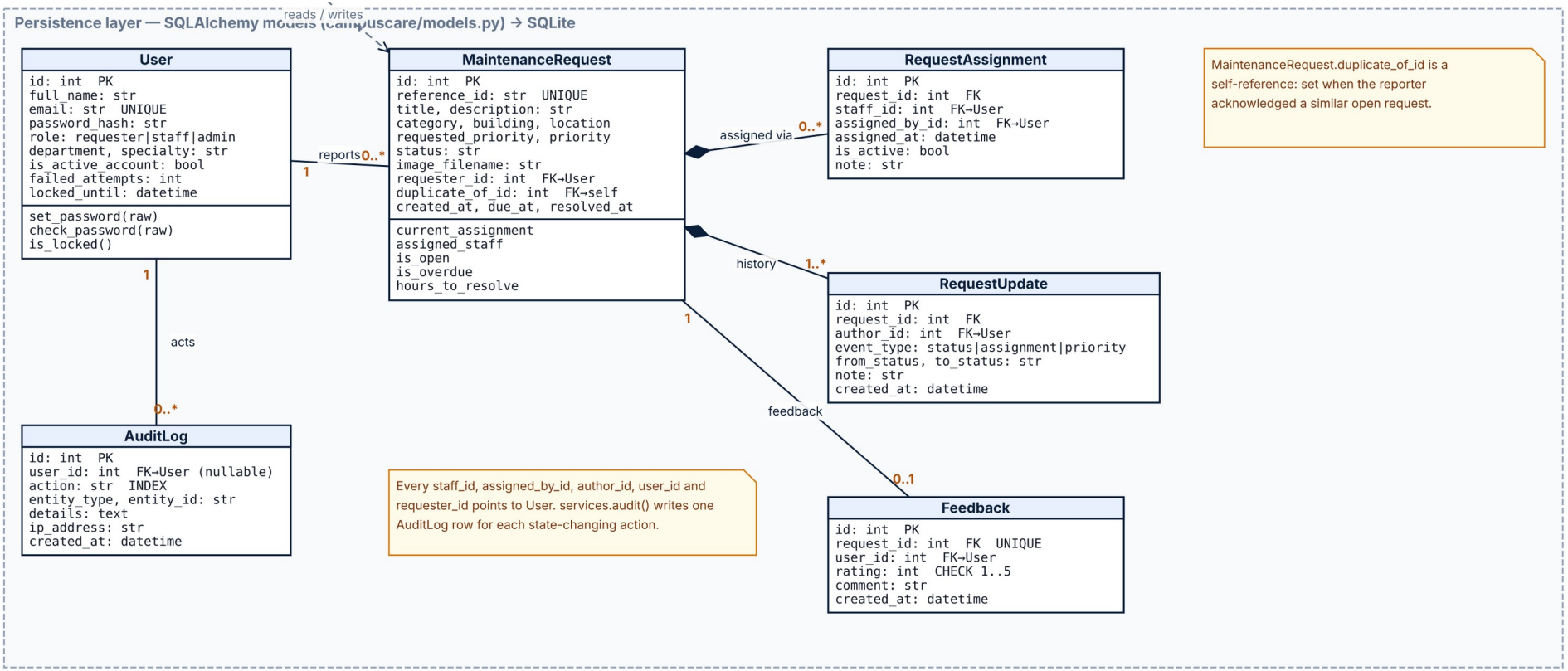


Figure 4: Class diagram (2/2): persistence layer

6.4 Sequence diagrams

Figures 5 and 6 show the submission and assignment workflow requested by the brief. Part A covers submission: the route hands the form to `create_request`, which validates the input, looks for duplicates and either raises `DuplicateFound` (the reporter sees a warning with links) or applies priority escalation and stores the request, its first history row and an audit entry in one commit. Part B covers the administrator starting the review and assigning the request: the service checks the transition table, the role and the ownership, then validates the chosen staff member before recording the assignment.

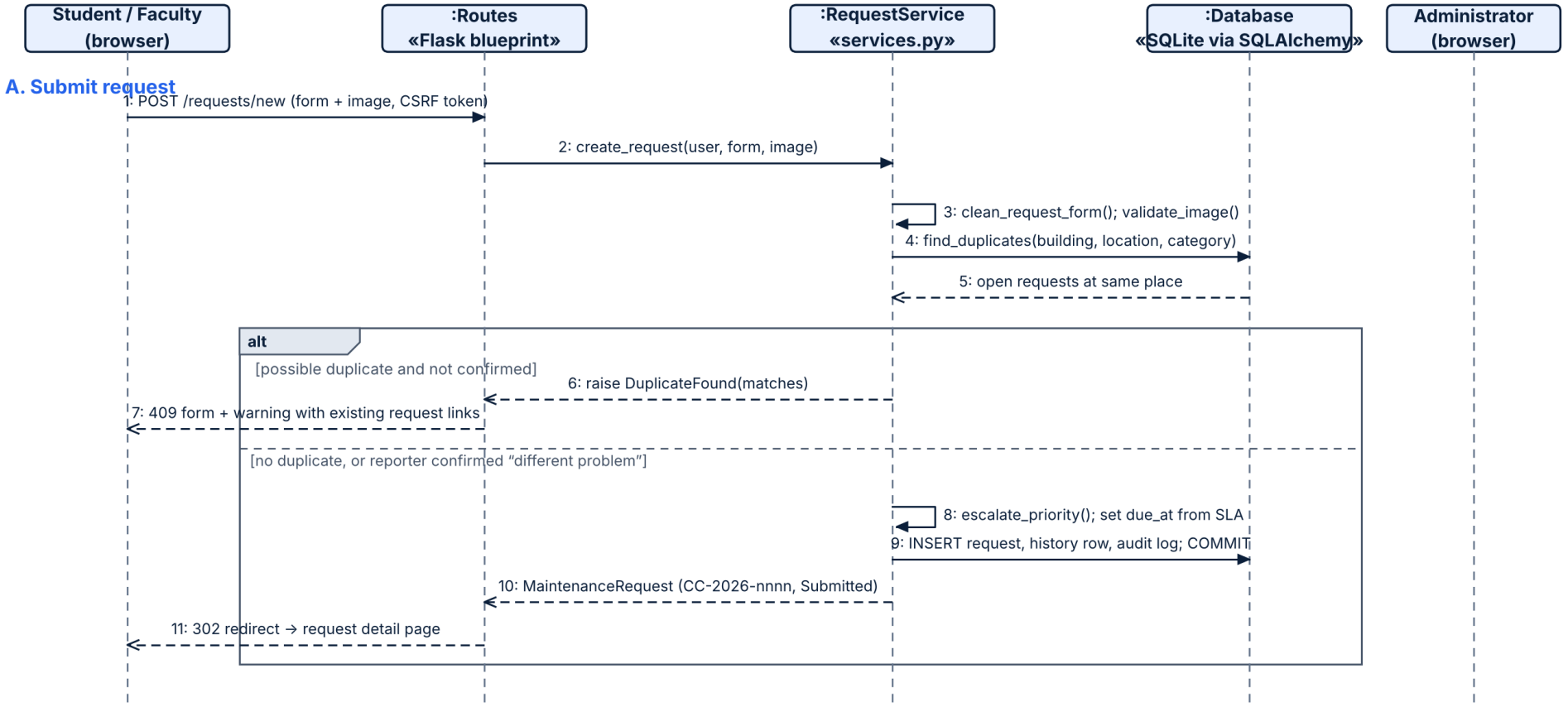


Figure 5: Sequence diagram (A): submitting a request, including the duplicate branch

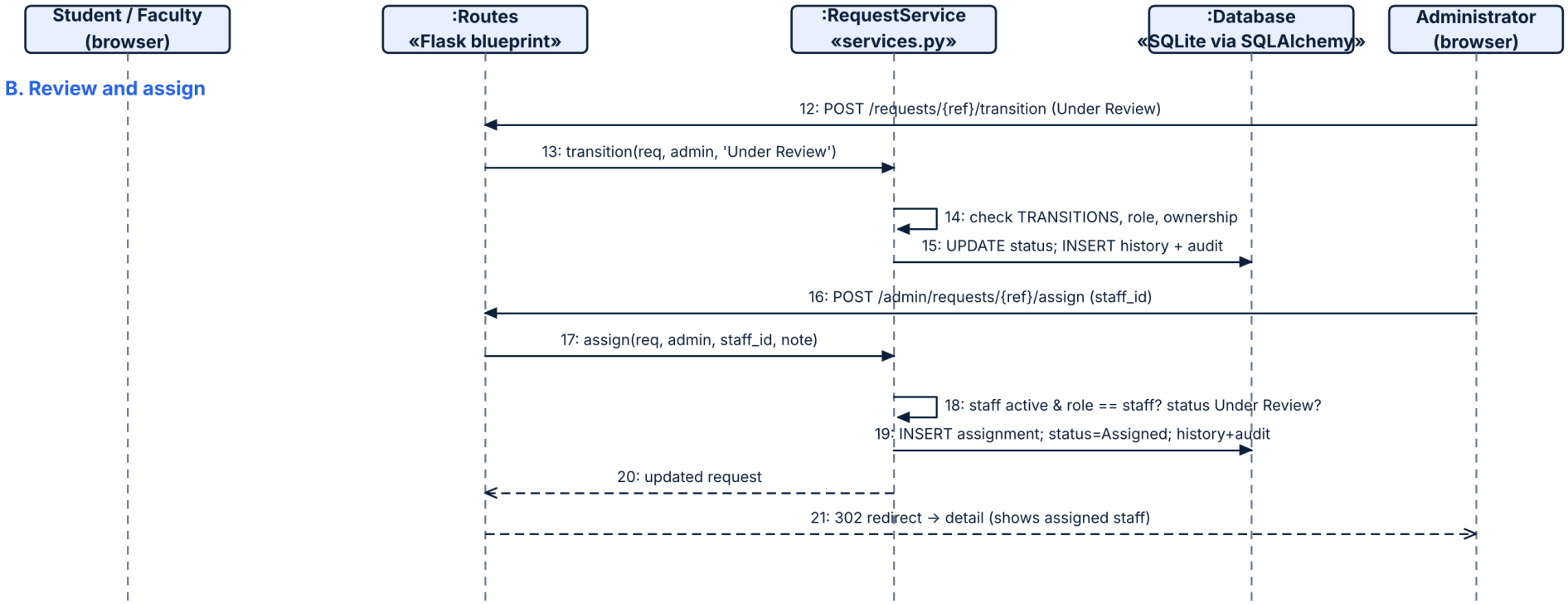


Figure 6: Sequence diagram (B): administrator review and assignment

6.5 Workflow state diagram

Figure 7 shows the request life-cycle. This diagram is a direct picture of the `TRANSITIONS` table in `services.py`. The role in brackets is who may perform the step; “Assigned” can only be reached by assigning to a staff member, which guarantees that every assigned request has an assignee. Rejected and Closed are terminal.

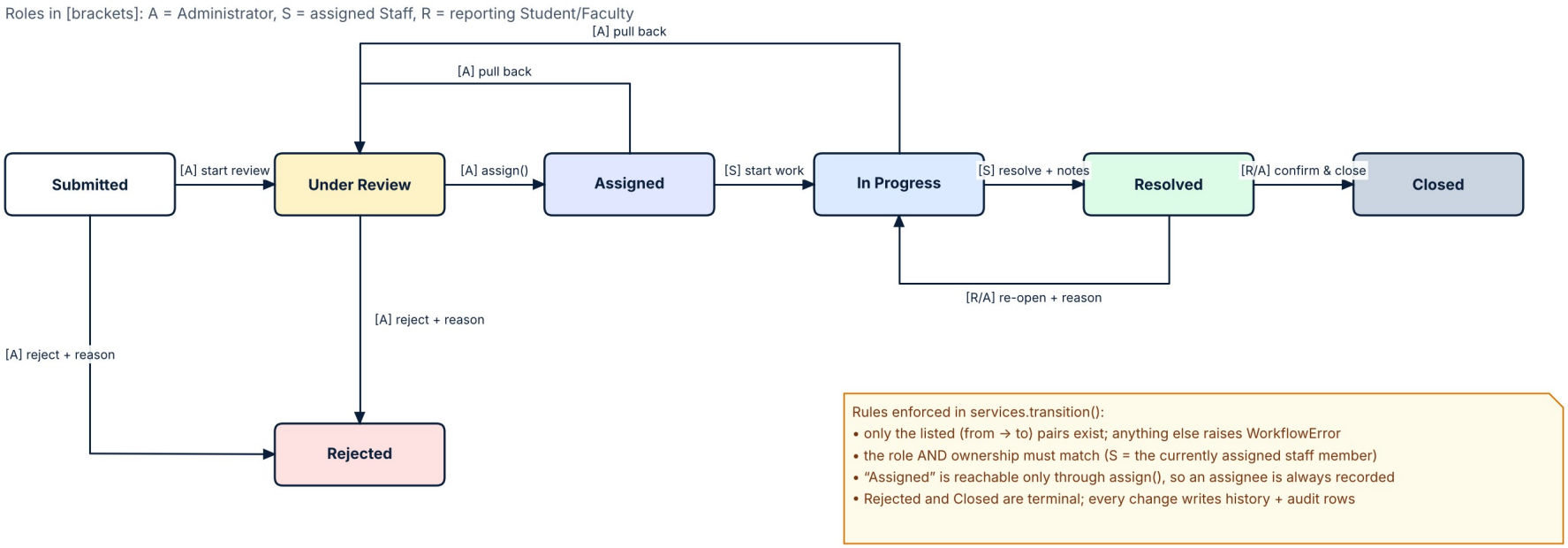


Figure 7: Request workflow state diagram

6.6 Database design

The schema has six tables in third normal form (Figure 8, Table 12). Reporter, staff and administrator details exist only in `users`. Assignments, status history and feedback are separate tables, so re-assigning or re-opening a request never overwrites earlier facts. `maintenance_requests` stores both `requested_priority` and `priority` so that automatic escalation remains visible. The current assignee is the single active row in `request_assignments`. The implemented models in `models.py` are the source of the diagram; the descriptions are repeated in `docs/DATABASE.md`.

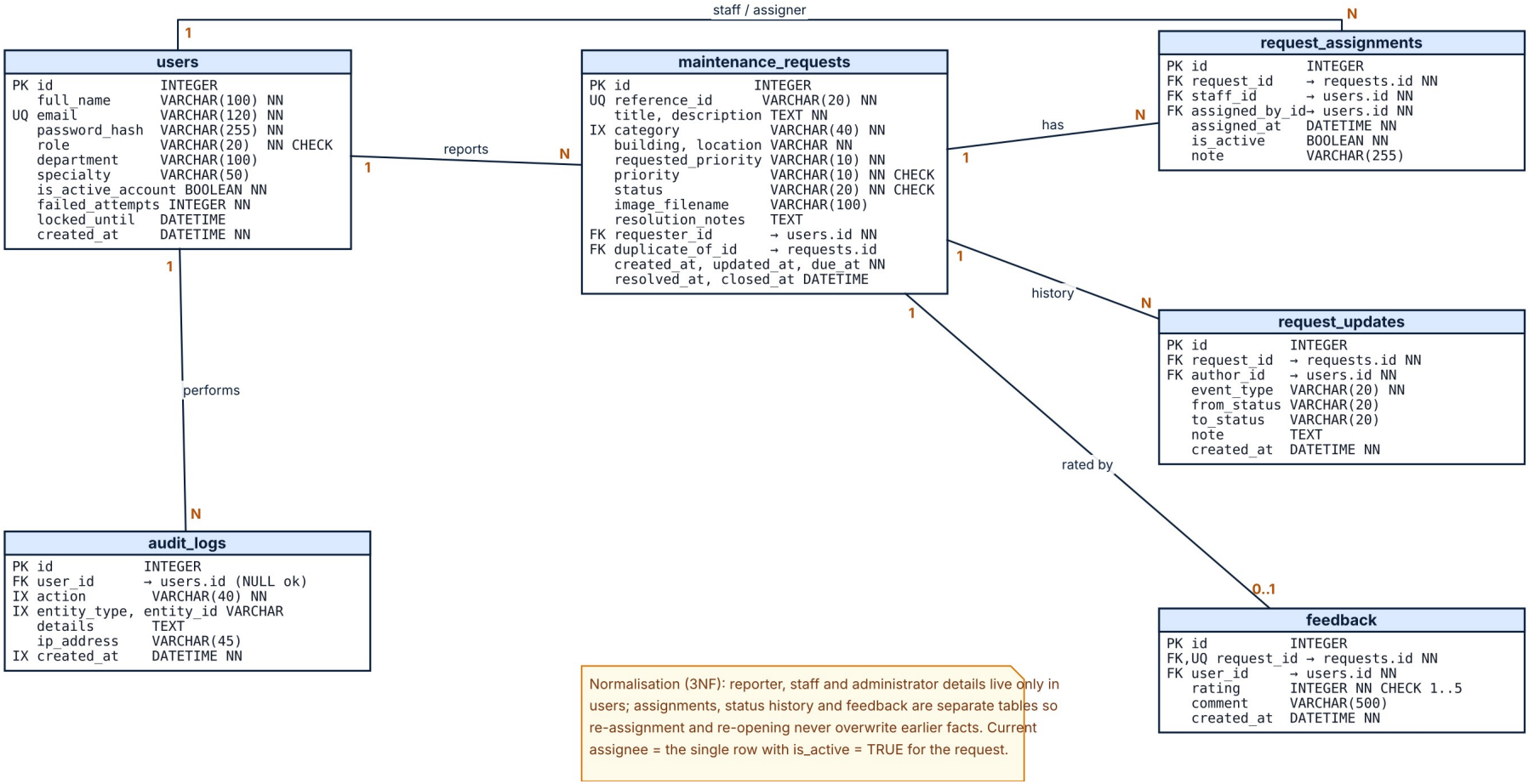


Figure 8: Entity–relationship diagram

Table 12: Tables, keys and constraints

Table	Keys and constraints	Indexes (reason)
users	PK id; UNIQUE email; CHECK role in (requester, staff, admin)	email (login lookup), role (staff lists)
maintenance_requests	PK id; UNIQUE reference_id; FK requester_id, duplicate_of_id; CHECK status, priority, requested_priority	(status, priority) for queues; (building, category, status) for duplicate search; category, requester_id
request_assignments	PK id; FK request_id, staff_id, assigned_by_id	request_id, staff_id (current assignee and workload)
request_updates	PK id; FK request_id, author_id	request_id (history display)
feedback	PK id; UNIQUE FK request_id; CHECK rating 1–5	unique request_id (one rating per request)
audit_logs	PK id; FK user_id (nullable for unknown e-mail)	action, created_at, (entity_type, entity_id)

A demonstration data set of 8 users and 15 requests in different states is created by `seed.py`. All names and e-mail addresses are fictional.

7. Implementation

7.1 Technology and structure

The prototype uses Python 3, Flask, Flask-SQLAlchemy, Flask-Login and Werkzeug on the server, SQLite for data, and HTML5, CSS3, ES6 JavaScript, Bootstrap 5 and Chart.js on the client. Bootstrap, Bootstrap Icons and Chart.js are copied into `static/vendor` so the application works without internet access. The code base is about 2,460 lines (roughly 1,900 lines of Python including tests, 430 lines of templates, plus CSS and JavaScript) and exposes 25 routes.

Table 13: Main modules

Module	Responsibility
app.py, config.py	Entry point and configuration (secret key, database URL, upload limits, lock-out settings, test configuration).
campuscare/models.py	SQLAlchemy models, constraints, derived properties such as `is_overdue` and `current_assignment`.
campuscare/services.py	All business rules: workflow, permissions, duplicate detection, priority escalation, assignment, feedback, statistics, reports, user creation, audit.
campuscare/routes/	Thin blueprints for authentication, reporter pages, staff pages and administrator pages; `role_required` decorator.
campuscare/templates, static	Role-aware layout, pages, CSS theme, form scripts, chart scripts.
seed.py	Creates the database and demonstration data (uses the same service functions as the application).
tests/	Pytest suite with a recording plugin that writes real results to `test-results/results.json`.

7.2 Key mechanisms

Workflow enforcement. The permitted transitions are data, not scattered conditions:

```
TRANSITIONS = {
  ("Submitted","Under Review"): {ADMIN},   ("Submitted","Rejected"): {ADMIN},
  ("Under Review","Assigned"): {ADMIN},    ("Under Review","Rejected"): {ADMIN},
  ("Assigned","In Progress"): {STAFF},     ("Assigned","Under Review"): {ADMIN},
  ("In Progress","Resolved"): {STAFF},     ("In Progress","Under Review"):
{ADMIN},
  ("Resolved","Closed"): {REQUESTER, ADMIN}, ("Resolved","In Progress"):
{REQUESTER, ADMIN},
}
```

`transition()` rejects unknown pairs, checks the role, then checks ownership (the reporter must own the request; staff must be the current assignee), then checks that required notes are present. The web interface uses `available_actions()` from the same table to decide which buttons to show, so the interface cannot offer something the server would refuse.

Duplicate detection. Among open requests with the same building and category, the new location is compared to existing ones after normalisation (lower case, punctuation removed); equal strings, or one containing the other, count as a match. The check runs live while the form is filled and again on submit, where an unconfirmed duplicate stops the submission.

Priority escalation. `escalate_priority()` raises the Safety & Security category to at least High and any text containing a danger keyword to Critical, ignoring the phrase “fire extinguisher”. The reporter’s choice stays in `requested_priority`; the history records why the priority changed. The due time is created from the effective priority.

Assignment support. `suggest_staff()` ranks active staff by specialty match and then by the number of open assigned tasks, and the administrator’s drop-down shows both, which is how limited resources are balanced without removing the administrator’s decision.

Security measures. Werkzeug password hashing; five failed logins lock an account for 15 minutes and every attempt is audited; the login error message is identical for unknown users, wrong passwords and locked accounts; all POST requests need a CSRF token; responses carry `Content-Security-Policy`, `X-Frame-Options` and `X-Content-Type-Options`; uploads are checked for extension, size (2 MB) and real file signature, and stored under random names in a directory that is not publicly served.

8. User Interface and Main Features

The interface uses a dark navy sidebar, white cards, blue accents and colour-coded status and priority badges. The sidebar changes with the role, so each of the three user types sees only its own navigation. All screenshots in this section were captured from the running application with a scripted browser session over the seeded demonstration data and the flow described in the captions; none are mock-ups.

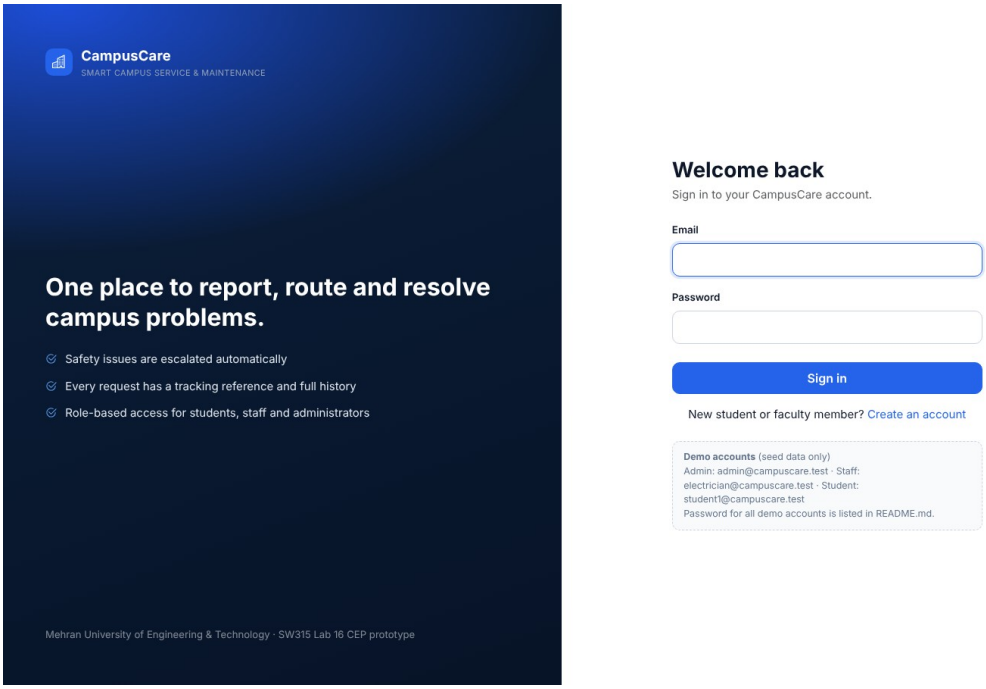


Figure 9: Login page

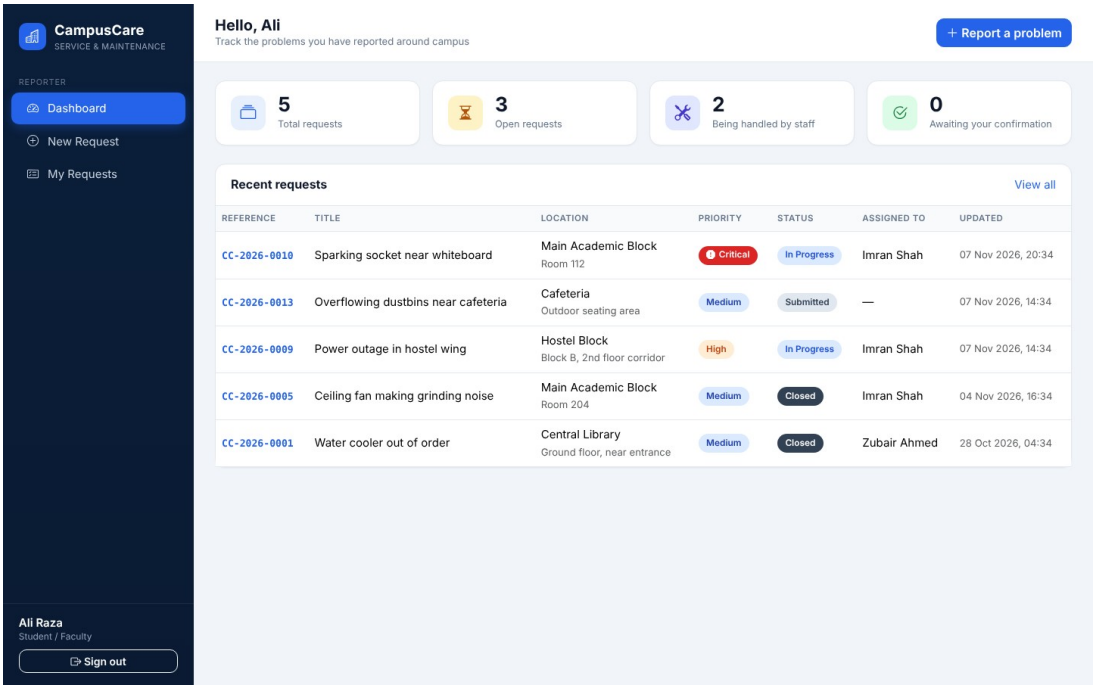


Figure 10: Student / faculty dashboard

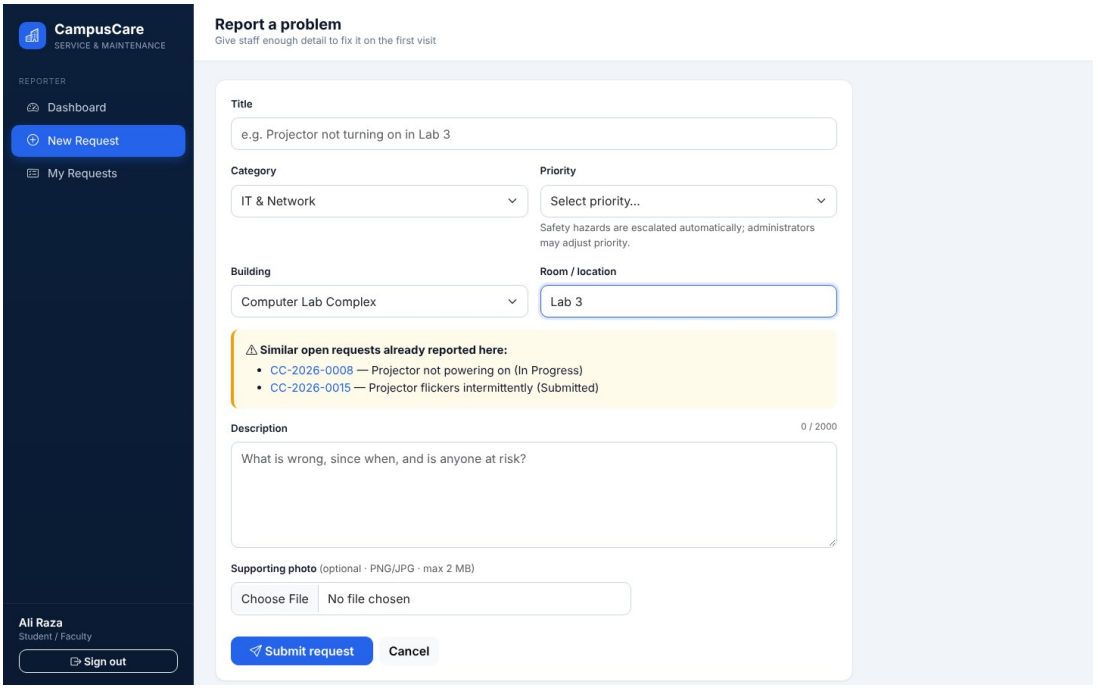


Figure 11: Request form showing the live duplicate hint for Lab 3

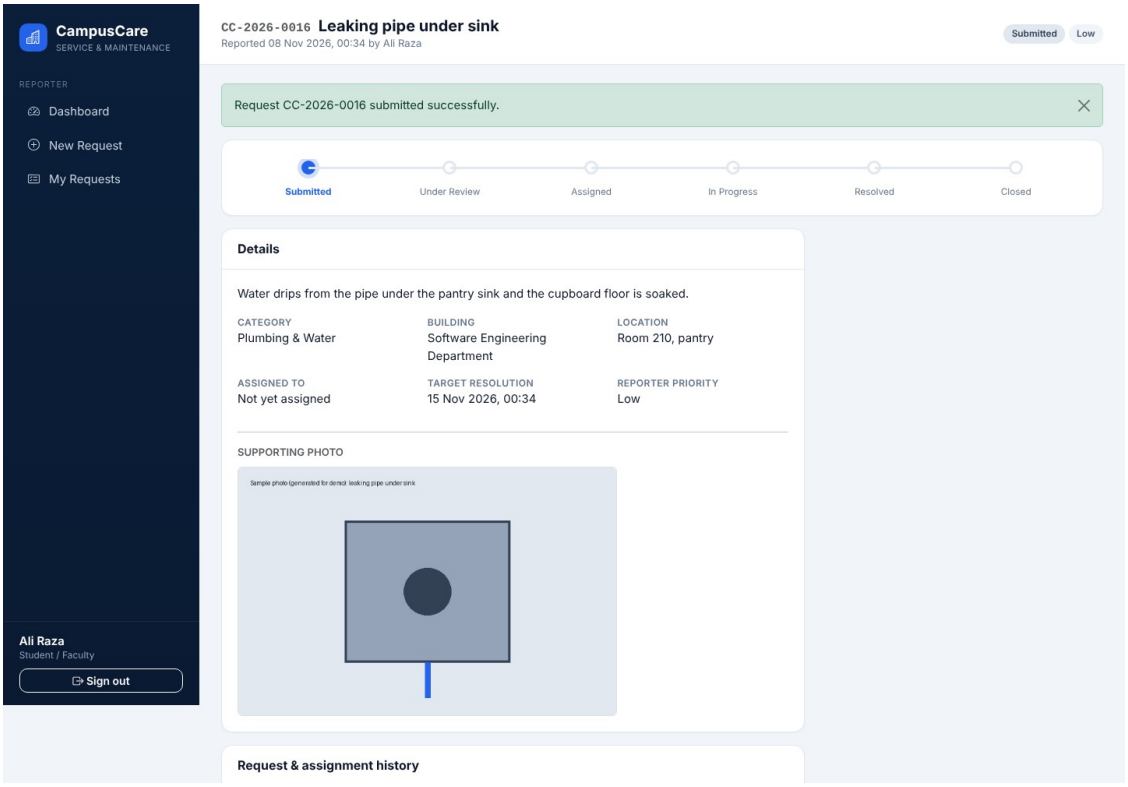


Figure 12: Request tracking page with status stepper, photo and history

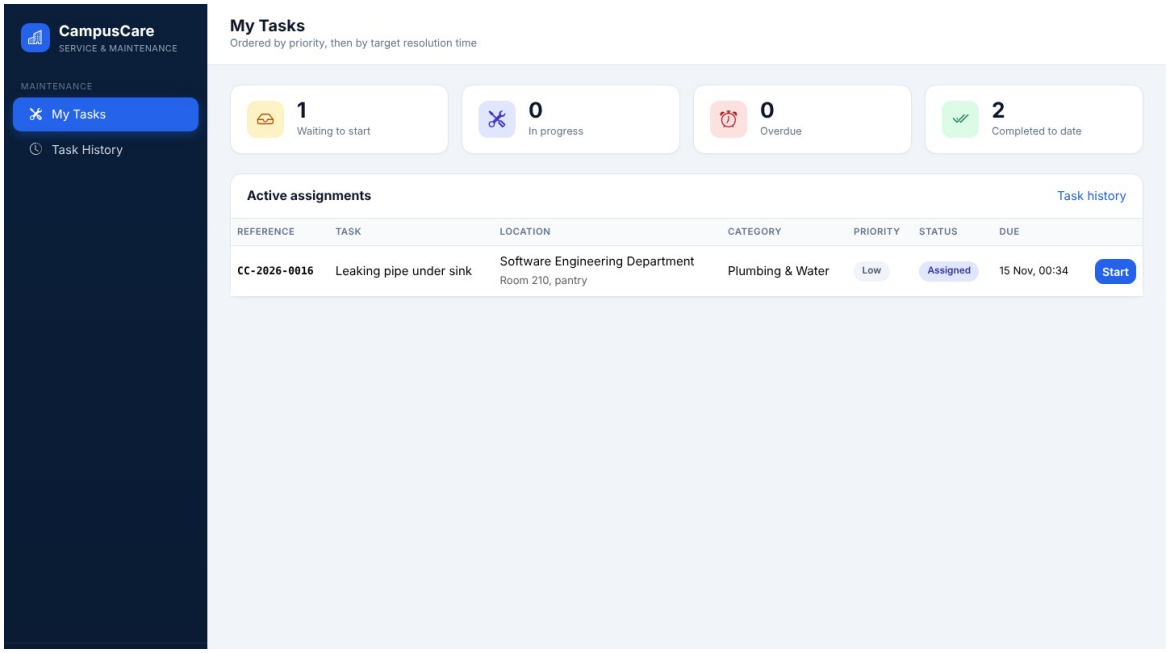


Figure 13: Maintenance staff dashboard (tasks ordered by priority and due time)

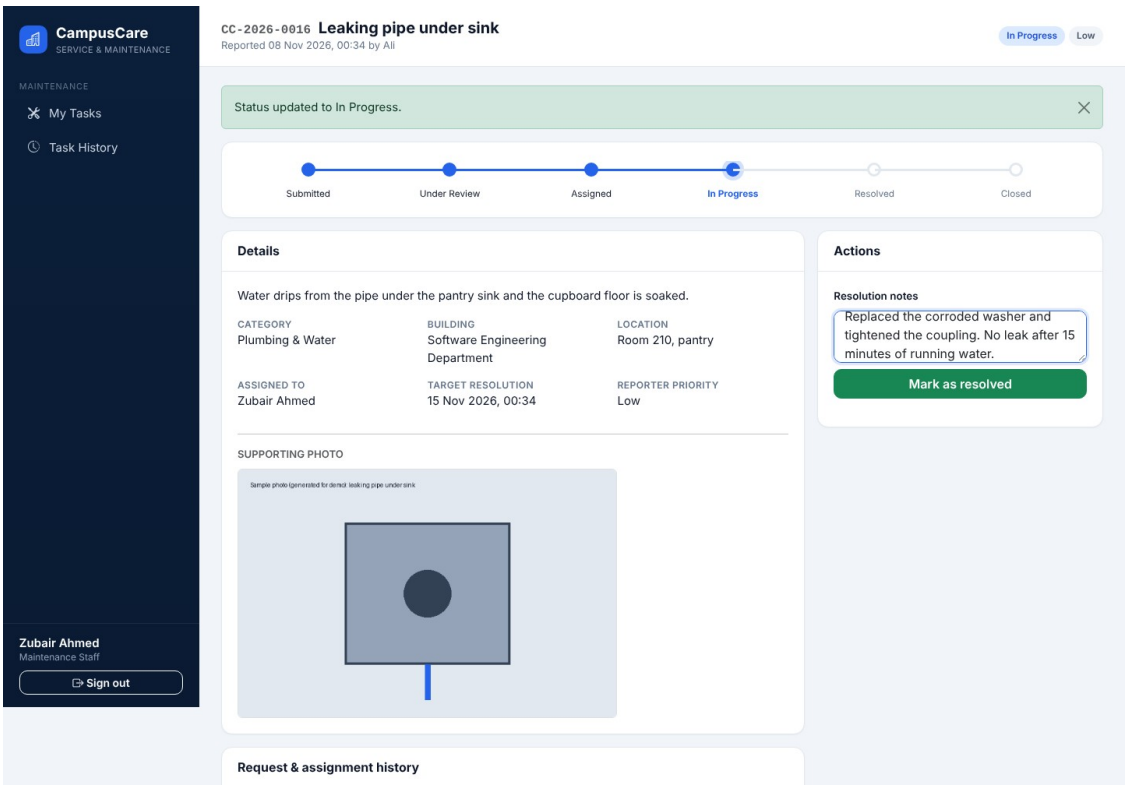


Figure 14: Staff resolving a task with resolution notes

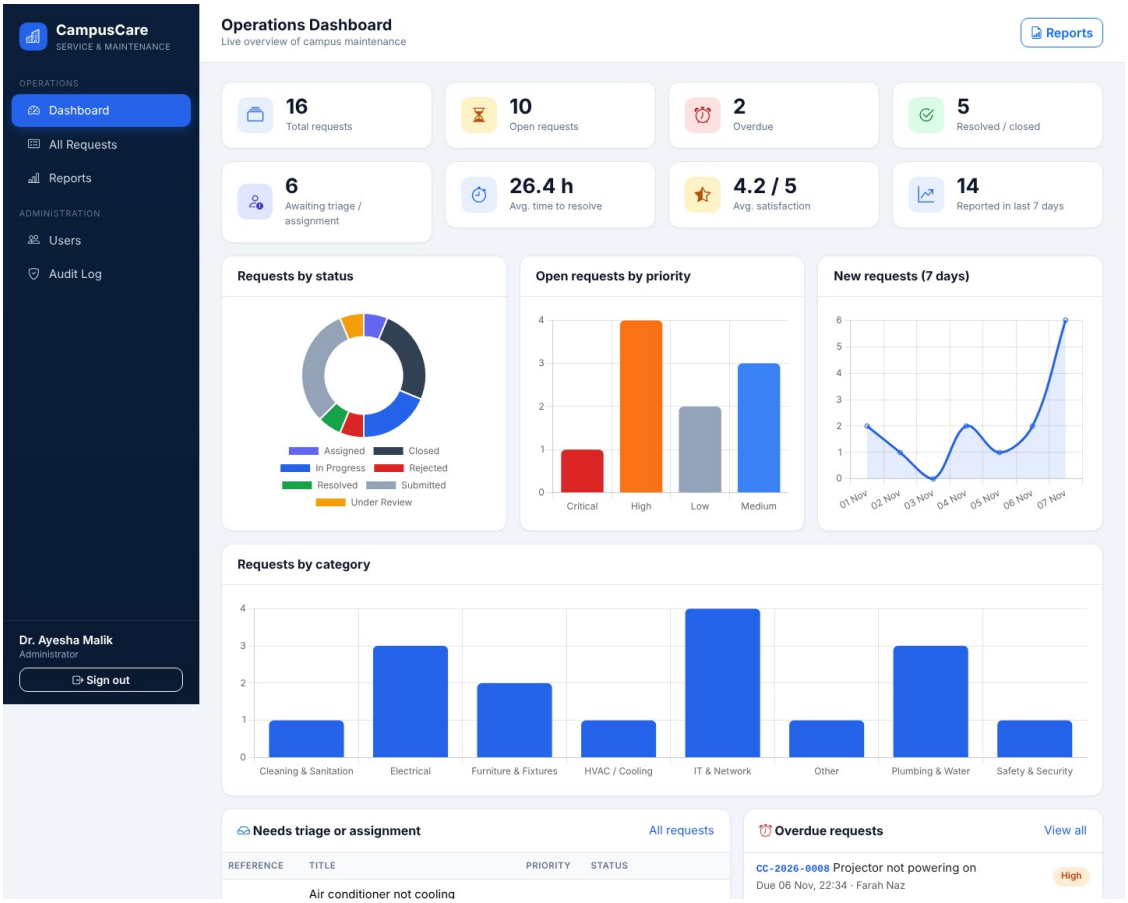


Figure 15: Administrator dashboard with statistics and Chart.js charts

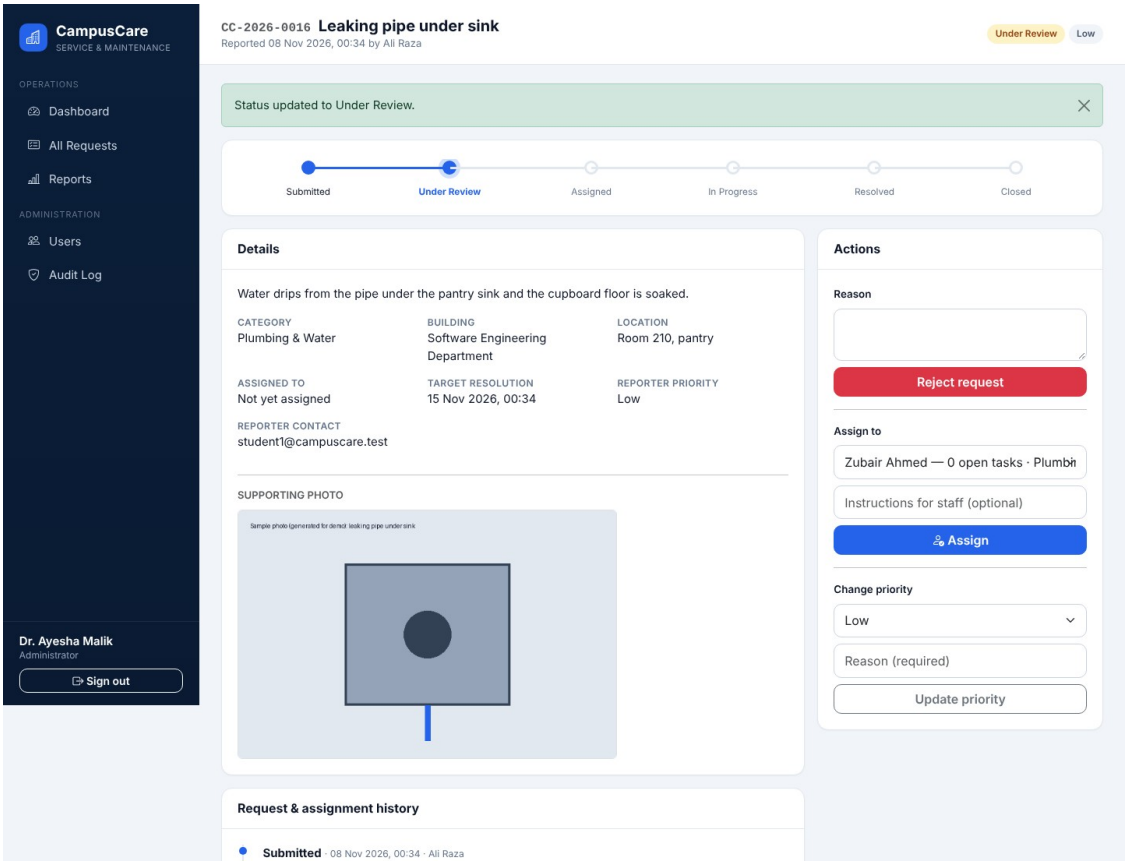


Figure 16: Administrator assignment interface with specialty and workload hints

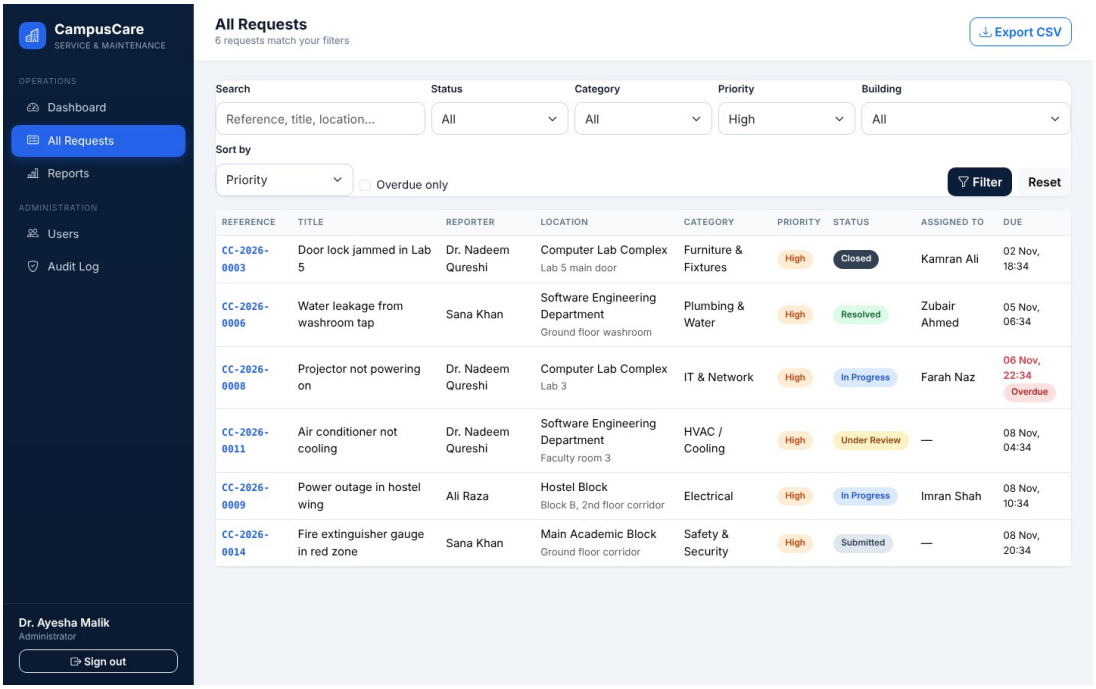


Figure 17: Administrator request list filtered by priority and sorted by priority

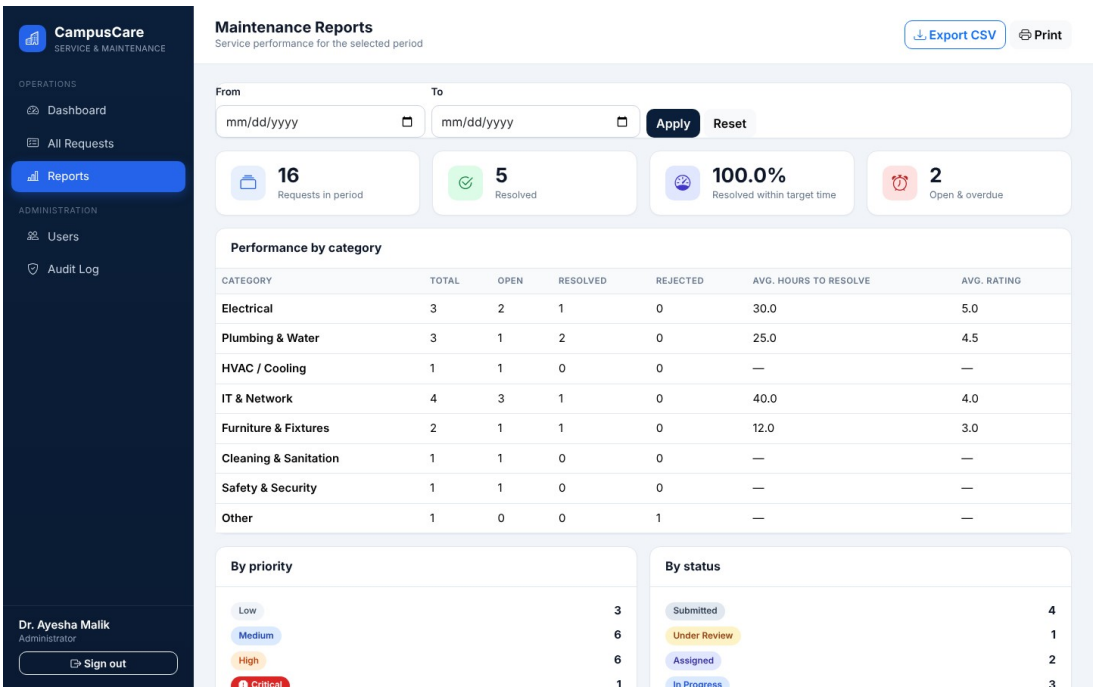


Figure 18: Maintenance report with performance by category

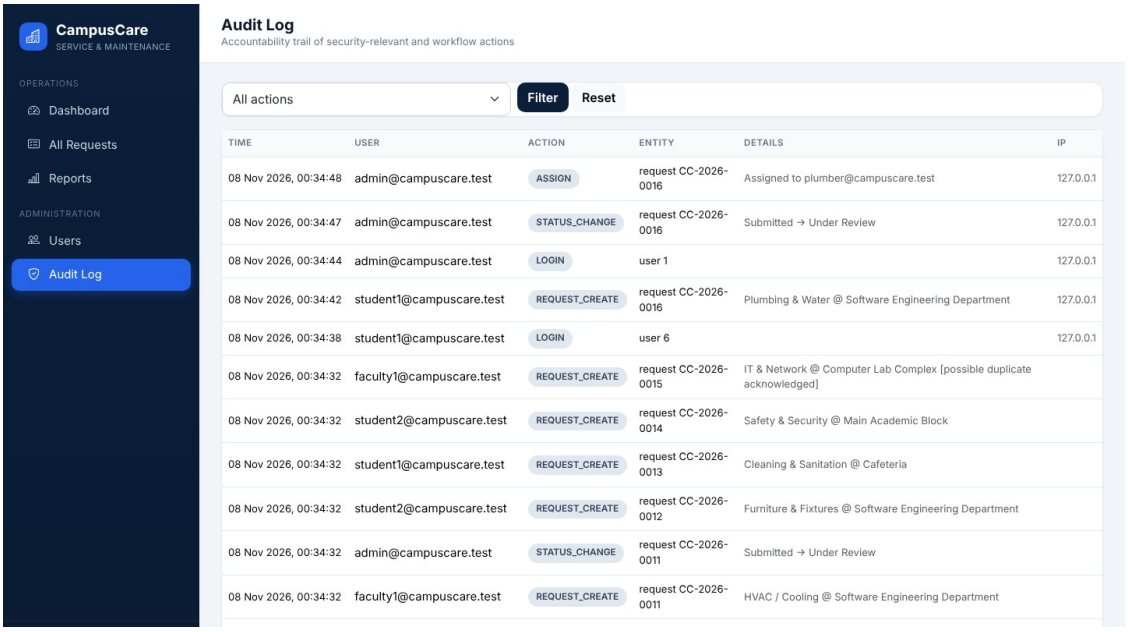


Figure 19: Audit log

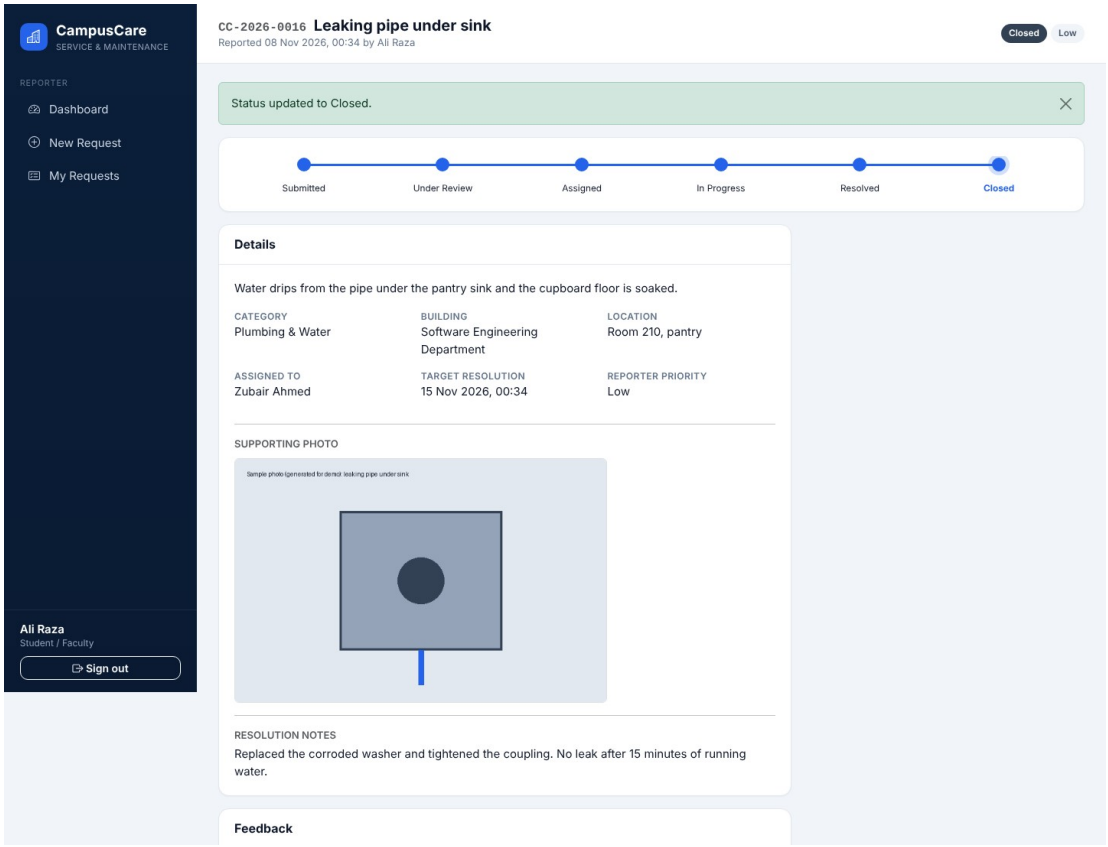


Figure 20: Closed request with full history

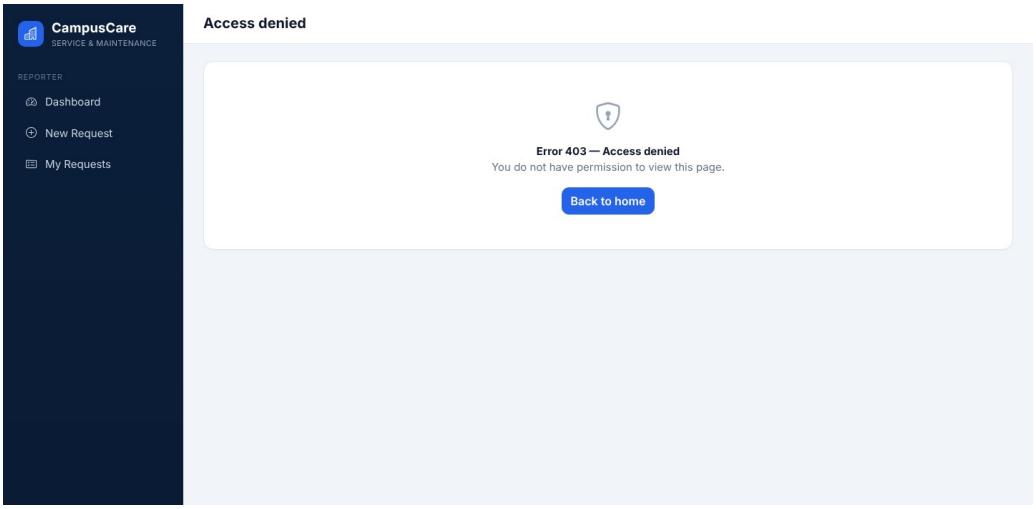


Figure 21: Unauthorised access: a student opening an administrator URL receives 403

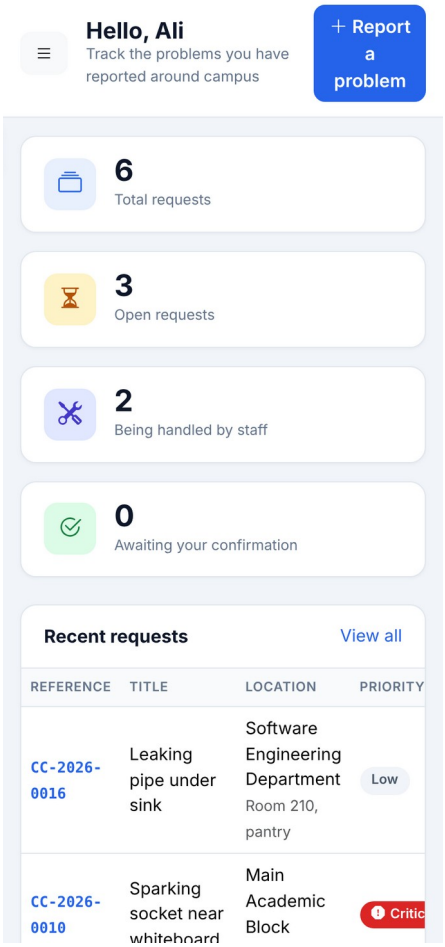


Figure 22: Responsive layout at phone width (390 px)

9. Development and DevOps

9.1 Version control

The project directory is structured for Git and contains a `.gitignore` (virtual environments, database, uploads, test output), `.env.example`, `requirements.txt` and a README with set-up instructions. **Repository status:** the repository has not been created yet, because the commit history must be produced by the group members themselves to be authentic.

`docs/DEVOPS.md` contains the initialisation commands and a suggested sequence of eleven commits.

Repository link: `[to be inserted by the group after pushing the project]`

9.2 Continuous integration

`.github/workflows/ci.yml` runs on every push to `main` and every pull request. It sets up Python 3.10 and 3.12, installs the dependencies, compiles all Python files as a build check, runs the full pytest suite, and uploads the machine-readable test results. The same two commands were run locally to produce the results in Section 10; the workflow itself has not yet run on GitHub, so no screenshot of a green run exists yet (see Appendix C).

9.3 Deployment considerations

For a real deployment the application should run under a WSGI server such as Gunicorn behind a reverse proxy with HTTPS, with `SECRET_KEY` and `DATABASE_URL` supplied as environment variables, a server database instead of SQLite, uploads on persistent storage, scheduled backups, and cookies marked `Secure`. These items are not implemented or tested here.

10. Testing and Evaluation

10.1 Methodology

We used automated pytest tests against an in-memory SQLite database that is rebuilt for every test. Tests work at two levels: service-level tests call the business functions directly (fast, precise, ideal for workflow rules), and HTTP-level tests use the Flask test client with real logins to check routing, permissions, CSRF, uploads and responses. Negative tests are as numerous as positive ones. A small pytest plugin records the outcome and an observed-result string for each test into `test-results/results.json`; the “actual result” columns below are taken from that file, not typed by hand. Some tests (TC-18) run with several inputs and are reported as one case.

10.2 Results

The final run executed 37 pytest items covering 35 numbered test cases; 35 of 35 cases passed and none failed (22.8 s). Table 14 gives the 15 most important cases in full; Appendix A lists all 35. Figure 23 shows the terminal output of the final run.

Table 14: Key test cases with expected and actual results

ID	Req.	Description / precondition	Input	Expected result	Actual result (recorded)	Status
TC-01	FR-01, FR-02	Valid login opens the role dashboard Pre: Student account exists	Correct e-mail and password	Redirect (302) to home; dashboard returns 200	HTTP 302, redirected to /	Pass
TC-02	FR-01, NFR-01	Wrong password is rejected with a generic message Pre: Student account exists	Correct e-mail, wrong password	HTTP 401; generic error; user still anonymous	HTTP 401; generic error shown; no session created	Pass

ID	Req.	Description / precondition	Input	Expected result	Actual result (recorded)	Status
TC-03	NFR-01	Account locks after repeated failures Pre: Student account exists	5 wrong passwords, then the correct password	Correct password refused while the account is locked	After 5 failures correct password gives HTTP 401; locked_until set=True	Pass
TC-04	FR-02, NFR-01	Self-registration cannot create a privileged role Pre: Anonymous visitor	Registration form with extra field role=admin	Account is created with role “requester”	Account created with role='requester' although role=admin was posted	Pass
TC-07	FR-02	Student cannot open administrator pages Pre: Student logged in	GET /admin/dashboard, /admin/users, /admin/reports, /admin/audit	HTTP 403 for all four	HTTP codes for 4 admin URLs: [403, 403, 403, 403]	Pass
TC-09	FR-02, NFR-06	Student cannot view another student’s request Pre: Request owned by student A; student B logged in	GET /requests/<ref>	HTTP 403	HTTP 403 for another user's request	Pass
TC-12	FR-03	Missing or too-short fields are rejected Pre: Student logged in	Empty title/category/building/location/priority, 5-character description	HTTP 422; messages shown; no row created	HTTP 422; rows created=0	Pass
TC-13	FR-10	Danger keyword escalates priority to Critical Pre: Student logged in	Category Electrical, priority Low, text mentions “sparks”	Effective priority Critical; due time 8 h after creation	Requested=Low, effective=Critical, due in 8 h	Pass
TC-15	FR-09	Duplicate request is detected and needs confirmation Pre: Open request exists for Lab 3 / IT & Network	Second request for “lab 3 ” in same building and category	DuplicateFound raised; confirmed copy stored with duplicate_of set	Duplicate warning listed CC-2026-0001; confirmed copy flagged duplicate_of=CC-2026-0001	Pass
TC-18	FR-03, NFR-01	Invalid uploads are rejected Pre: Student logged in	(a) .exe file, (b) text file named .png, (c) PNG data named .jpg	HTTP 422 each time; no request created	evil.exe: HTTP 422; requests created=0 | note.png: HTTP 422; requests created=0 | a.jpg: HTTP 422; requests created=0	Pass
TC-20	FR-05, FR-06, FR-15	Admin starts review, then assigns to staff Pre: Submitted request; admin logged in	Transition to Under Review; POST assign with staff_id and note	Status Assigned; assignment, history and audit rows written	HTTP 302; status=Assigned; assigned_to=Test Electrician; history=3; audit ASSIGN=1	Pass

ID	Req.	Description / precondition	Input	Expected result	Actual result (recorded)	Status
TC-21	FR-05	Invalid transitions are refused Pre: Submitted request; admin logged in	Submitted→Closed, Submitted→Assigned, Submitted→Resolved	WorkflowError each time; status unchanged; message shown	Submitted→Closed, Submitted→Assigned, Submitted→Resolved all refused; status still Submitted; message shown=True	Pass
TC-23	FR-05, FR-07	Staff cannot act on another member's task Pre: Task assigned to staff A; staff B exists	Staff B tries Start work and Resolve	PermissionDenied both times	Unassigned staff blocked from Start and Resolve with PermissionDenied	Pass
TC-28	FR-11	Feedback only after resolution, once, valid rating, by owner Pre: Assigned request	Early feedback; rating 9; other student; duplicate feedback	All four refused; valid rating 4 stored	Early, out-of-range, foreign and duplicate feedback refused; stored rating=4	Pass
TC-29	FR-08	Filters and search return the expected subset Pre: Three requests of mixed category/priority	category=Electrical; priority=Low; both; q=Fan	2, 2, 1 and 1 rows respectively	category=Electrical→2; priority=Low→2; both→1; search 'Fan'→1	Pass

```
Terminal — python -m pytest -v

============================= test session starts ==============================
platform linux -- Python 3.12.3, pytest-9.2.0, pluggy-1.7.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/claude/CampusCare
plugins: platformdirs-4.13.0
collecting ... collected 37 items
test_tc01_valid_login_redirects_to_role_dashboard          PASSED [  2%]
test_tc02_invalid_password_rejected_with_generic_message   PASSED [  5%]
test_tc03_account_locks_after_repeated_failures            PASSED [  8%]
test_tc04_self_registration_cannot_create_privileged_role  PASSED [ 10%]
test_tc05_weak_password_rejected_on_registration           PASSED [ 13%]
test_tc06_anonymous_user_redirected_to_login               PASSED [ 16%]
test_tc07_student_cannot_open_admin_pages                  PASSED [ 18%]
test_tc08_staff_cannot_manage_users_or_assign              PASSED [ 21%]
test_tc09_student_cannot_view_another_students_request     PASSED [ 24%]
test_tc10_csrf_token_required_when_enabled                 PASSED [ 27%]
test_tc11_valid_request_created_with_reference_and_history PASSED [ 29%]
test_tc12_missing_required_fields_rejected                 PASSED [ 32%]
test_tc13_safety_keyword_escalates_to_critical             PASSED [ 35%]
test_tc14_safety_category_minimum_priority_high_but_exting PASSED [ 37%]
test_tc15_duplicate_detected_and_blocked_until_confirmed   PASSED [ 40%]
test_tc16_non_duplicate_in_other_location_accepted         PASSED [ 43%]
test_tc17_valid_png_upload_stored_and_access_controlled    PASSED [ 45%]
test_tc18_invalid_uploads_rejected[evil.exe-MZ000000000000 PASSED [ 48%]
test_tc18_invalid_uploads_rejected[note.png-this is not an image] PASSED [ 51%]
test_tc18_invalid_uploads_rejected[a.jpg-\x89PNG\r\n\x1a\n PASSED [ 54%]
test_tc19_oversized_image_rejected                         PASSED [ 56%]
test_tc20_admin_review_then_assign_records_assignment_and_ PASSED [ 59%]
test_tc21_invalid_transition_rejected_at_service_and_http_ PASSED [ 62%]
test_tc22_assignment_requires_review_and_valid_staff       PASSED [ 64%]
test_tc23_staff_cannot_resolve_task_assigned_to_someone_el PASSED [ 67%]
test_tc24_resolution_requires_notes                        PASSED [ 70%]
test_tc25_full_lifecycle_through_web_interface             PASSED [ 72%]
test_tc26_reject_requires_reason_and_is_terminal           PASSED [ 75%]
test_tc27_reopen_resolved_request_needs_explanation        PASSED [ 78%]
test_tc28_feedback_only_after_resolution_once_and_valid_ra PASSED [ 81%]
test_tc29_filters_return_expected_subset                   PASSED [ 83%]
test_tc30_overdue_detection_and_priority_sorting           PASSED [ 86%]
test_tc31_admin_priority_change_requires_reason_and_resets PASSED [ 89%]
test_tc32_report_summary_and_csv_export                    PASSED [ 91%]
test_tc33_suggest_staff_prefers_specialty_then_lowest_work PASSED [ 94%]
test_tc34_deactivated_user_cannot_login                    PASSED [ 97%]
test_tc35_admin_cannot_demote_or_deactivate_self           PASSED [100%]
============================== 37 passed in 22.77s ==============================
```

Figure 23: Terminal output of the final test run (python -m pytest -v)

10.3 Interpretation

All 35 cases pass in the final run, but the first complete run did not: 34 of 37 items passed and 3 failed. Two of the failures (TC-17 and TC-25) were caused by the test set-up, not by the application: our fixture kept one Flask application context open for the whole test, so Flask-Login cached the first logged-in user and several test clients behaved as the same person. The third failure (TC-29) was a counting mistake in the test, which counted each reference twice because it appears in both a link and its text. We fixed the test code (isolating the client and counting table rows) and left the application unchanged. We report this because it shows the value of the negative tests: the access-control assertions in TC-17 failed loudly when the test harness was wrong. To be sure the issue was not hiding a real flaw, the screenshot session used separate browser contexts per role and confirmed that a student receives 403 on an administrator URL (Figure 21).

10.4 Security testing considerations

Automated tests cover authentication failure and lock-out, privilege escalation through registration, role-based access, object-level access to requests and uploaded images, CSRF, and upload validation. They do not replace a security review: we did not test session fixation, brute-force limits across many accounts, HTTP header behaviour behind a proxy, file-parsing

exploits, SQL injection beyond relying on the ORM's parameter binding, or cross-site scripting beyond Jinja2's automatic escaping.

10.5 Limitations, failure risks and improvements

Table 15: Limitations and recommended improvements

Limitation / failure risk	Consequence	Recommended improvement
Keyword-based escalation (RISK-01)	False positives overload the Critical queue; non-English or misspelt hazards (e.g. Urdu text) are missed.	Add a configurable keyword list, Urdu and Roman-Urdu terms, and review the false-positive rate on real requests.
Location matching is string-based (RISK-06)	Reports such as "Lab 3" and "Computer Lab three" are not matched.	Replace free text with a managed list of rooms per building.
No notifications (RISK-07)	Reporters and staff must open the site to see changes; a Critical request could wait unseen.	E-mail or push alerts for assignment, resolution and overdue Critical items.
SQLite and one process (RISK-02)	Write contention and no horizontal scaling.	PostgreSQL, several workers, connection pooling.
Lock-out is per account	An attacker can lock out a known user (denial of service).	Add per-IP throttling and an administrator unlock.
Uploads are validated, not re-encoded or scanned	A crafted valid image could still carry unexpected content.	Re-encode images on upload; add antivirus scanning.
Automated tests do not drive the browser	Layout and chart errors are found only by manual review.	Add browser-based tests for the main journeys.

10.6 Tests not executed

Table 16: Checks that were not performed

Check	Status
Load and concurrency testing	Not executed. Only a single-user timing check was made (Section 11).
Cross-browser and accessibility (screen reader, contrast audit)	Not executed. Pages were reviewed in Chromium only.
GitHub Actions run	Not executed yet; the workflow exists but the repository has not been pushed.
Penetration test / dependency vulnerability scan	Not executed.
Usability test with real users	Not executed.

11. System Evaluation

Measured on the development container with the 16-request demonstration data set and 30 repeated requests per page through the test client (so the figures include template rendering but not network time): administrator dashboard 7.3 ms, request list 7.1 ms, reports 6.6 ms, request detail 5.1 ms and the statistics API 5.4 ms on average. These numbers show that nothing in the

prototype is accidentally slow; they say nothing about behaviour with thousands of requests or concurrent users, which we did not test.

Against the requirements: all fifteen functional requirements are implemented and each has at least one automated test, except FR-04 and FR-12, which are verified by logic tests and screenshots only. NFR-01 and NFR-06 are covered by tests; NFR-02 (usability) was judged by inspection and has not been tested with users; NFR-03 was measured only as described above; NFR-04, NFR-05 and NFR-07 are properties of the code and schema.

The design choices that gave the most value for their size were the single transition table (it removed duplicated permission logic and made the tests simple), the separate assignment and history tables (the audit view and re-assignment came almost free) and the requested/effective priority split (it made the escalation rule explainable). The weakest parts are the keyword heuristic and the free-text location, both listed as risks.

12. Compliance with Lab 16 and Assessment Criteria

Table 17: Lab 16 requirement checklist

Lab 16 requirement	Evidence	Section	Status
Problem statement / SRS; ≥3 FR, ≥2 NFR, ≥2 constraints or risks, ≥2 stakeholder groups	15 FR, 7 NFR, 4 CR, 7 RISK, 4 stakeholder groups	3, 4	Done
≥2 engineering alternatives, comparison, justified selection and trade-off	Approach A vs B, 9-criterion table, trade-off, smaller decisions	5	Done
Use case, class and sequence diagrams, consistent with requirements	Use case, class, sequence, architecture, state and ER diagrams; Draw.io sources	6	Done
Working prototype	Flask application, 25 routes, screenshots	7, 8	Done
Source code on GitHub/GitLab with meaningful commits	Repository-ready project and commit plan	9	Group action required: push and generate real commits
Automated build/test or described DevOps pipeline	GitHub Actions workflow (not yet run on GitHub)	9	Done (workflow written); run evidence pending
≥5 test cases, positive and negative, expected/actual/pass-fail	35 cases, 37 test items, actual results recorded	10, App. A	Done
Interpretation, limitation and improvement	Sections 10.3–10.5	10	Done
Screenshots / evidence of diagrams, features, repository, tests	22 screenshots, 2 terminal captures, 6 diagrams	6, 8, 10	Repository and Actions screenshots pending
Constraints, risks, alternatives, reason for design, lessons learned	Sections 4, 5, 14	4, 5, 14	Done

Table 18: Evidence for the five assessment criteria

Criterion	Where the evidence is
R1 Constraints, requirements, stakeholder demands	Sections 2–4: CEP analysis, stakeholder conflicts with design responses, numbered FR/NFR/CR/RISK, traceability matrix.
R2 Originality and contribution	Problem-specific decisions: requested versus effective priority with safety escalation, target resolution times, duplicate detection with reporter override, workload-aware assignment suggestions, privacy-limited staff view, access-controlled photos (Sections 3, 5.3, 7.2).
R3 Engineering knowledge, design and standards	Layered architecture, state machine, normalised schema with constraints, UML models that match the code (Sections 5–6); ISO/IEC/IEEE 29148 structure and OWASP-based security measures.
R4 Efficiency and practical implementation	Working prototype with 25 routes, seed data, CI workflow, vendored libraries, timing measurements (Sections 7–9, 11).
R5 Testing, evaluation and reporting	35 recorded test cases with actual results, an honest account of the first failing run, limitations, risks, untested areas and improvements (Section 10).

13. Limitations and Future Improvements

- **Notifications and integration:** e-mail or push notifications and single sign-on with university accounts were left out; they are the most valuable next step because they close the loop with reporters.
- **Scale:** a server database, several workers, caching of dashboard statistics (currently computed per request in Python) and pagination of the dashboard queries are needed beyond a department-size deployment.
- **Smarter triage:** a managed room list, tunable keyword lists including Urdu, and (with real data) a classifier that proposes priorities for an administrator to confirm.
- **Staff features:** shift calendars, parts and cost tracking, and mobile-friendly task views.
- **Quality:** browser-based tests, accessibility review, dependency scanning and a proper penetration test.

14. Lessons Learned and Team Contributions

14.1 Lessons learned

- Making the workflow a table first paid off repeatedly: the permissions, the buttons in the interface, the state diagram and the tests all came from one source.
- Stakeholder conflicts are easier to resolve when each one becomes a visible rule with a constant (target hours, minimum priority) than when it stays an opinion.
- Heuristics need examples that break them. The “fire extinguisher” case, found while writing tests, changed the escalation rule.
- A failing test is not always a failing product: our first run showed two harness problems and one test bug, and we fixed the tests without changing the application. Reading failures before “fixing” them avoided hiding a real defect.
- Generating the diagrams and the report tables from the same data as the code (the diagram script, the recorded test results) keeps documents consistent with the implementation.
- We underestimated how much of the effort goes into evidence: screenshots, repository history and CI runs must be produced by doing the work, not at the end.

14.2 Team and proposed responsibilities

The allocation below is a **proposed division of responsibilities**. It has not been confirmed against the actual commit history, and the group should correct it so that it states what each member really did before submission.

Table 19: Group members and proposed responsibilities

Member	Roll no.	Proposed responsibility
Shariyar	24SW028	Requirements and problem analysis: SRS, stakeholder analysis, CEP characteristics, traceability matrix.
Saqib Anwar	24SW046	Backend: authentication, RBAC, workflow service and transition rules.
Bansi	24SW052	Frontend: templates, CSS theme, form scripts, dashboard charts.
Safiullah	24SW107	Database design: models, constraints, indexes, seed data, ER diagram.
Saifullah	24SW137	UML and architecture: use case, class, sequence and state diagrams; engineering alternatives.
Janak	24SW155	Testing, DevOps and integration: pytest suite, CI workflow, repository, final report assembly.

15. Conclusion

CampusCare shows that a modest, well-structured Flask application can address a real campus problem while exposing the engineering questions behind it: who may do what, how scarce capacity is shared, how safety overrides convenience, and how accountability is kept without exposing personal data. The group compared a monolith with a distributed design and chose the monolith for scope, security simplicity and time, accepting limits on scale that are documented. The prototype implements the full request life-cycle for three roles, is covered by 35 recorded test cases that pass in the final run, and is packaged with a CI workflow. Three items remain for the group: pushing the project to a repository with genuine commits, running the CI workflow once, and confirming the contribution table. The main technical weaknesses, keyword-based escalation and free-text locations, are identified with concrete improvements.

References

[1] ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering.

[2] Object Management Group, Unified Modeling Language Specification, version 2.5.1, 2017.

[3] OWASP Foundation, OWASP Top 10:2021 and OWASP Application Security Verification Standard 4.0.

[4] Pallets Projects, Flask documentation, https://flask.palletsprojects.com.

[5] SQLAlchemy documentation, https://docs.sqlalchemy.org.

[6] Sommerville, I., Software Engineering, 10th ed., Pearson, 2016.

[7] Fowler, M., Patterns of Enterprise Application Architecture, Addison-Wesley, 2002.

[8] Department of Software Engineering, MUET Jamshoro, SW315 Lab 16 — Open-Ended Lab: Complex Engineering Problem (CEP), 2026.

Appendix A: All Test Cases and Recorded Results

Table 20: All test cases (results from test-results/results.json of the final run)

ID	Req.	Test	Actual result (recorded)	Status
TC-01	FR-01, FR-02	Valid login opens the role dashboard	HTTP 302, redirected to /	Pass
TC-02	FR-01, NFR-01	Wrong password is rejected with a generic message	HTTP 401; generic error shown; no session created	Pass
TC-03	NFR-01	Account locks after repeated failures	After 5 failures correct password gives HTTP 401; locked_until set=True	Pass
TC-04	FR-02, NFR-01	Self-registration cannot create a privileged role	Account created with role='requester' although role=admin was posted	Pass
TC-05	FR-01, NFR-01	Weak password rejected at registration	HTTP 422; user created=False	Pass
TC-06	FR-02	Anonymous user is redirected to the login page	HTTP 302 → /login?next=%2Fdashboard	Pass
TC-07	FR-02	Student cannot open administrator pages	HTTP codes for 4 admin URLs: [403, 403, 403, 403]	Pass
TC-08	FR-02, FR-06	Staff cannot manage users or assign requests	/admin/users → 403; POST assign → 403	Pass
TC-09	FR-02, NFR-06	Student cannot view another student’s request	HTTP 403 for another user's request	Pass
TC-10	NFR-01	POST without CSRF token is refused	POST without CSRF token → HTTP 400	Pass
TC-11	FR-03, FR-15	Valid request is stored with reference ID and history	HTTP 302; ref=CC-2026-0001; status=Submitted; history rows=1	Pass
TC-12	FR-03	Missing or too-short fields are rejected	HTTP 422; rows created=0	Pass
TC-13	FR-10	Danger keyword escalates priority to Critical	Requested=Low, effective=Critical, due in 8 h	Pass
TC-14	FR-10	Safety category has minimum priority High	Requested=Low, effective=High	Pass
TC-15	FR-09	Duplicate request is detected and needs confirmation	Duplicate warning listed CC-2026-0001; confirmed copy flagged duplicate_of=CC-2026-0001	Pass
TC-16	FR-09	Same category in a different room is not a duplicate	Different room accepted: CC-2026-0002, duplicate_of=None	Pass
TC-17	FR-03, NFR-01	Valid PNG stored under random name; access controlled	Stored as random name 642bb594562d4f4aaa78fa8a50241a2c.png; owner → 200; other student → 403	Pass
TC-18	FR-03, NFR-01	Invalid uploads are rejected	evil.exe: HTTP 422; requests created=0 | note.png: HTTP 422; requests created=0 | a.jpg: HTTP 422; requests created=0	Pass
TC-19	FR-03, NFR-01	Oversized image rejected	2 MB+ image: HTTP 422	Pass
TC-20	FR-05, FR-06, FR-15	Admin starts review, then assigns to staff	HTTP 302; status=Assigned; assigned_to=Test Electrician; history=3; audit ASSIGN=1	Pass
TC-21	FR-05	Invalid transitions are refused	Submitted→Closed, Submitted→Assigned, Submitted→Resolved all refused; status still Submitted; message shown=True	Pass

ID	Req.	Test	Actual result (recorded)	Status
TC-22	FR-06	Assignment needs Under Review status and a real staff user	Assign refused while Submitted; refused for a student account; status unchanged	Pass
TC-23	FR-05, FR-07	Staff cannot act on another member's task	Unassigned staff blocked from Start and Resolve with PermissionDenied	Pass
TC-24	FR-07	Resolution requires meaningful notes	Note 'ok' refused; full note accepted; status=Resolved; resolved_at set=True	Pass
TC-25	FR-04 – FR-07	Full lifecycle through the web interface	Final status Closed; path: Submitted → Under Review → Assigned → In Progress → Resolved → Closed	Pass
TC-26	FR-05	Reject needs a reason and is terminal	Reject without reason refused; with reason accepted; no transitions out of Rejected	Pass
TC-27	FR-05	Re-opening a resolved request needs an explanation	Re-open accepted with reason; status=In Progress; resolved_at cleared=True	Pass
TC-28	FR-11	Feedback only after resolution, once, valid rating, by owner	Early, out-of-range, foreign and duplicate feedback refused; stored rating=4	Pass
TC-29	FR-08	Filters and search return the expected subset	category=Electrical→2; priority=Low→2; both→1; search 'Fan'→1	Pass
TC-30	FR-08, FR-10, FR-12	Overdue detection and priority sorting	Priority order: ['Critical', 'Low']; overdue list: ['CC-2026-0001']	Pass
TC-31	FR-08, FR-10	Priority change needs a reason and resets the due time	Priority Low→High; due moved from +168h to +24h	Pass
TC-32	FR-13	Report summary and CSV export	Report: total=2, resolved=1; CSV HTTP 200, 2 data rows, type=text/csv	Pass
TC-33	FR-06	Staff suggestion prefers specialty, then lowest workload	Ranking for Electrical: [('Test Electrician', True, 0), ('Test Plumber', False, 0)]	Pass
TC-34	FR-01, FR-14	Deactivated account cannot sign in	Deactivated account login → HTTP 401	Pass
TC-35	FR-14	Administrator cannot demote or deactivate self	Own role after attempt: admin; active=True	Pass

Appendix B: Running the Project

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python seed.py --reset      # demo data; password for all demo users: Demo@1234
python app.py               # http://127.0.0.1:5000
python -m pytest -v         # automated tests
python tools/make_uml.py    # regenerate diagrams (needs matplotlib, pillow)
```

Demo accounts: admin@campuscare.test; electrician@, plumber@, it.support@, general@campuscare.test (staff); student1@, student2@, faculty1@campuscare.test (reporters). Database evidence from the screenshot session is shown below.

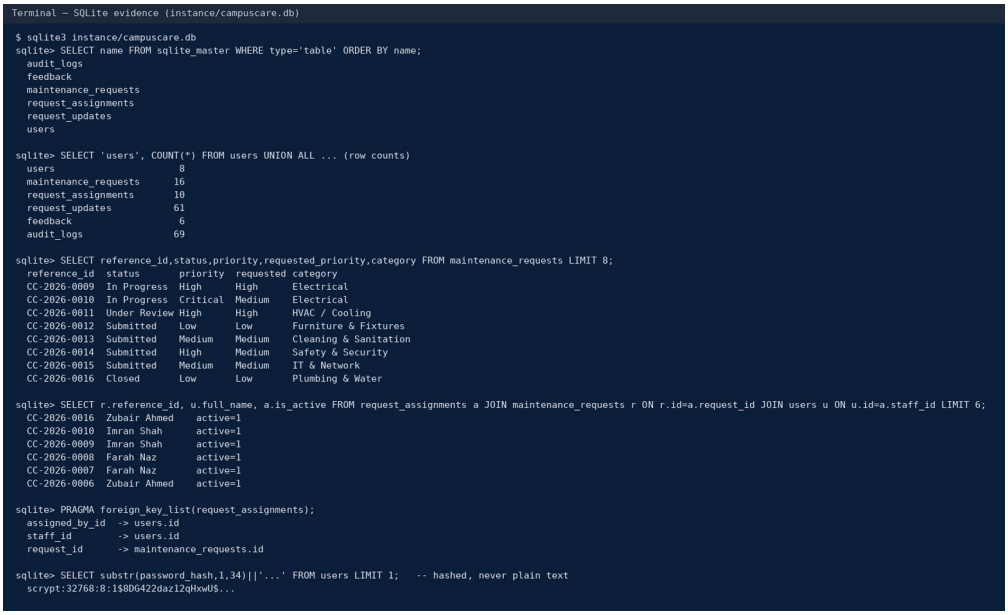


Figure 24: SQLite evidence: tables, row counts, sample rows, foreign keys, hashed password

Appendix C: Evidence the Group Must Still Produce

Table 21: Missing evidence checklist

#	Evidence	How to produce it
1	Repository URL and commit history	Create the repository, follow docs/DEVOPS.md, make real commits from each member's account; screenshot the Commits page.
2	Green GitHub Actions run	Push to main; open the Actions tab after the workflow finishes; screenshot it.
3	Confirmed contribution table	Compare Section 14.2 with the commit history and edit the table.
4	Instructor name and submission details on the cover	Fill in the placeholder on the cover page.
5	Rename the file	SCD_Open-EndedLab_<Names>_<RollNos>.pdf before submitting.